

WAREHOUSE INVENTORY

SQL ANALYSIS REPORT

From database preparation to decision-ready reporting

Project Title: Warehouse Inventory SQL Analysis

Database / Table: warehouse_inventory.warehouse_transactions

SQL Platform Used: MySQL 8.0

Prepared By: [Your Full Name]

Submission Date: 17 August 2026

2. Executive Summary

Executive Summary

This report analyses 5,000 warehouse inventory transactions covering 1 January 2025 to 16 May 2026. Overall, the operation received 2,614,361 units, dispatched 1,353,875 units and recorded an inventory loss rate of 5.73%. The strongest overall performer was Leeds Fulfillment, based on the highest estimated gross profit (£49.83M) combined with the lowest inventory loss percentage (5.52%) of any warehouse. The highest operational risk was concentrated in Liverpool Logistics, which recorded the highest loss percentage (6.03%) and the highest audit failure rate (18.11%) of any location, alongside above-average delivery times. Estimated gross profit across the operation was £362.3M, while inventory losses were valued at approximately £89.75M — equivalent to 24.8% of estimated gross profit. The single largest operational risk identified was dispatch reliability: only 29.84% of transactions were dispatched on time, while 40.82% were recorded as Delayed. Priority actions are to (1) investigate the causes of widespread dispatch delay across all locations, (2) review handling, storage and audit practices at Liverpool Logistics, Bristol Supply Hub and Newcastle Depot, the three warehouses flagged with both above-average loss and above-average delivery time, and (3) engage Prime Wholesale UK, the only supplier combining above-average supply volume with above-average loss percentage, on quality control.

Key Figures at a Glance

Metric	Value
Total transactions	5,000
Transaction period	1 Jan 2025 – 16 May 2026 (17 months)
Total units received	2,614,361
Total units dispatched	1,353,875
Total remaining stock (estimated)	1,110,658 units
Overall inventory loss percentage	5.73%
Overall return percentage	4.64%
Overall audit failure rate	15.42%
Overall on-time dispatch rate	29.84%
Average delivery days	5.53 days
Total estimated sales value	£1,170,728,632.87
Total dispatched inventory cost	£808,422,287.40
Total estimated gross profit	£362,306,345.47
Total inventory loss value	£89,754,534.13

3. Business Problem

Warehouse transaction data records what happened at each stage of the inventory lifecycle — receipt, storage, handling, dispatch and return — but does not by itself explain why performance differs across locations, products, suppliers or operational teams. A warehouse with a high received quantity may still carry hidden damage, missing units, delayed dispatches, audit failures or customer returns; equally, a high sales value does not guarantee strong profitability if inventory loss is significant.

This report addresses that gap by connecting inventory volume, stock movement, quality, delivery, audit, returns and financial measures into a single operational view. Throughout the analysis, a clear distinction is maintained between large absolute values (which reflect scale) and poor rates (which reflect efficiency or risk) — a large warehouse can record the highest number of damaged units while still maintaining a lower damage percentage than a smaller warehouse.

Central Analytical Question

Central Question

How effectively is inventory being received, stored, handled, dispatched and converted into revenue, and where are losses, delays, audit failures and returns creating operational or financial risk?

Decisions This Report Supports

Business Area	How the Analysis Supports Decisions
Inventory control	Identifies excess stock, low stock movement, damage, missing units and possible stock inconsistencies.
Warehouse operations	Compares throughput, loss rates, delivery performance, audit results and return levels by warehouse.
Supplier management	Measures supply volume, loss rates, audit outcomes and product quality associated with each supplier.
Product management	Determines which products and categories generate volume, revenue, profit, returns and inventory risk.
Workforce management	Evaluates handling volume, loss exposure, delivery outcomes and audit performance associated with each employee.
Financial control	Estimates sales value, inventory cost, gross profit, loss value and return-related value.
Planning	Reveals monthly and yearly patterns that can support stock, staffing and dispatch planning.

4. Project Objectives

The following analytical objectives were completed in sequence, so that data reliability was established before business conclusions were produced:

- Confirm that the database structure and data types support accurate SQL analysis.
- Convert the transaction date from text in DD-MM-YYYY format into a valid DATE field.
- Detect duplicates, missing values, invalid categories and inconsistent quantity relationships.
- Establish overall KPIs before comparing individual business segments.
- Measure inventory inflow, outflow, remaining stock and loss units.
- Calculate loss, return, dispatch and audit rates using appropriate denominators.
- Estimate revenue, cost, gross profit and inventory loss value.
- Compare warehouse, product, supplier, employee and storage-section performance.
- Analyse monthly and yearly trends across the complete transaction period.
- Use advanced SQL to rank entities, compare records with averages and identify unusual behaviour.
- Convert query outputs into concise findings, implications and recommendations.

5. Dataset Overview

The dataset comprises 5,000 warehouse inventory transaction records across 22 fields, covering the period from 1 January 2025 to 16 May 2026 (confirmed by the converted `Transaction_Date` field, with zero conversion failures). It spans 8 warehouses, 8 UK cities, 5 product categories, 23 individual products, 8 suppliers, 8 storage sections and 8 employees handling inventory.

Attribute	Project Value
Number of records	5,000 transaction rows
Number of fields	22 columns
Transaction period	1 January 2025 to 16 May 2026
Warehouse coverage	8 warehouse names
Product coverage	5 categories and 23 products
Supplier coverage	8 suppliers
Inventory directions	Inbound and Outbound
Dispatch categories	On Time, Slightly Delayed, Delayed
Audit categories	Passed and Failed
Temperature-control flag	Yes and No

Role of Each Data Group

- Dimensions (`Warehouse_Name`, `City`, `Product_Category`, `Product_Name`, `Supplier_Name`, `Storage_Section`, `Employee_Handling`, `Batch_Number`) provide the segments used throughout the comparative analysis.
- Volume measures (`Quantity_Received`, `Quantity_Dispatched`, `Damaged_Units`, `Missing_Units`, `Customer>Returns`) drive the operational KPIs and loss calculations.
- Financial measures (`Unit_Cost_GBP`, `Selling_Price_GBP`) combine with volume measures to produce all estimated financial figures.
- Operational status fields (`Dispatch_Status`, `Delivery_Days`, `Stock_Audit_Status`, `Temperature_Controlled`) support delivery, compliance and storage-condition analysis.
- `Transaction_Date` (after conversion from text) underpins all time-series and trend analysis.

6. Database Preparation

6.1 Import Check

The raw dataset was loaded into a staging table (`warehouse_transactions_raw`) with 22 columns matching the data dictionary. A row count check confirmed all 5,000 records imported successfully, and a column count check against `information_schema` confirmed all fields were present. `DESCRIBE` output confirmed each numeric column (`Quantity_Received`, `Quantity_Dispatched`, `Damaged_Units`, `Missing_Units`, `Delivery_Days`, `Customer>Returns`) imported as `INT` and each price field (`Unit_Cost_GBP`, `Selling_Price_GBP`) imported as `DECIMAL(10,2)`, consistent with the handbook's recommended data types.

6.2 Text-Date Issue and Conversion Process

`Transaction_Date` was imported as text in DD-MM-YYYY format (e.g. '14-02-2025'). Text dates of this kind sort alphabetically rather than chronologically and cannot be reliably analysed with `YEAR`, `MONTH`, `QUARTER` or date-difference functions, so a valid `DATE` field was a prerequisite for all time-series analysis.

Following the handbook's data-safety requirement, the original text field was preserved throughout the conversion process rather than being modified in place:

- A new column, `Transaction_Date_Converted`, was added to the staging table using `STR_TO_DATE(Transaction_Date_Text, '%d-%m-%Y')`.
- A side-by-side comparison of the original text and converted date was reviewed on a 20-row sample.
- A validation query checked for rows where the text date was present but the converted date was null: zero such rows were found.
- `MIN()` and `MAX()` on the converted field confirmed the expected range of 1 January 2025 to 16 May 2026 exactly.
- The final analytical table (`warehouse_transactions`) was built from the validated data, with the converted field renamed `Transaction_Date` (`DATE`, `NOT NULL`) and the original text retained as `Transaction_Date_Text` for audit purposes.

6.3 Recommended Data Types Applied

The final table structure matches the handbook's data dictionary: `VARCHAR` for identifiers and categorical text, `INTEGER` for whole-unit counts, `DECIMAL(10,2)` for currency fields, and `DATE` for the converted transaction date. A primary key was applied to `Transaction_ID` (confirmed unique, zero duplicates) and indexes were added on `Warehouse_Name`, `Product_Category`, `Supplier_Name` and `Transaction_Date` to support the grouped and time-based queries used throughout this analysis.

7. Data Quality Assessment

Before any performance analysis was produced, the dataset was screened for completeness, uniqueness, validity and cross-field consistency. The consolidated results are summarised below.

Check	Result	Decision
Duplicate Transaction_ID values	0	No action required — Transaction_ID is a reliable unique key.
Null values (all columns)	0	No action required — dataset is fully populated.
Blank / whitespace-only text values	0	No action required.
Negative values (quantities, prices, delivery days, returns)	0	No action required.
Zero values in quantity/price fields	0	No action required.
Categorical values outside known lists	0	All categories match expected lists exactly.
Quantity_Dispatched > Quantity_Received	0	No stock-control inconsistency of this type present.
Damaged + Missing > Quantity_Received	3 rows	Retained and flagged for operational review; not deleted, as the volume is immaterial (0.06% of records) and does not distort aggregate KPIs.
Customer_Returns > Quantity_Dispatched	9 rows	Retained and flagged; represents 0.18% of records and is treated as a genuine exception list for operational follow-up rather than a data error.
Selling_Price_GBP < Unit_Cost_GBP	0	No loss-making list price rows present.
Delivery_Days outside 1-10 day range	0	All delivery times fall within the expected operational range.
Date conversion failures	0	All 5,000 text dates parsed successfully to DATE.
Batches linked to multiple products/suppliers	See appendix output	A small number of batch numbers are associated with more than one product or supplier; documented for traceability, not treated as an error.

Warehouse-City Relationship

A cross-field consistency check (task 2.12) tested whether each warehouse is associated with a single, fixed city. The result shows every one of the 8 warehouses is linked to all 8 cities in the dataset. This is not treated as a data-quality defect: it indicates that City records the transaction's destination or service area rather than the warehouse's own physical location, and this distinction is carried through to all geography-related interpretation in this report.

Overall Assessment

The dataset is of high quality. With zero duplicates, zero missing values and zero invalid categories, and only a small, clearly bounded set of cross-field exceptions (12 rows total, 0.24% of the dataset), no records were removed from the analytical table. All 5,000 transactions were retained for the performance analysis that follows.

8. Overall KPI Summary

These headline figures establish the scale and general condition of the operation before performance is segmented by warehouse, product, supplier or time period.

8.1 Volume KPIs

Metric	Value
Total transactions	5,000
Total units received	2,614,361
Total units dispatched	1,353,875
Average received per transaction	522.87 units
Average dispatched per transaction	270.78 units
Total remaining stock (estimated)	1,110,658 units

8.2 Quality and Risk KPIs

Metric	Value
Total damaged units	99,324
Total missing units	50,504
Total inventory loss units (damaged + missing)	149,828
Overall inventory loss percentage	5.73%
Total customer returns	62,822
Overall return percentage	4.64%
Overall audit failure rate	15.42%

8.3 Delivery KPIs

Metric	Value
Average delivery days	5.53 days
Minimum / maximum delivery days	1 day / 10 days
Overall on-time dispatch rate	29.84%
Delayed transaction share	40.82%
Slightly delayed transaction share	29.34%

8.4 Financial KPIs (Estimates)

Metric	Value (GBP)
Total estimated sales value	£1,170,728,632.87
Total dispatched inventory cost	£808,422,287.40
Total estimated gross profit	£362,306,345.47
Total inventory loss value	£89,754,534.13
Estimated return value (units returned × selling price)	£54,904,102.21

Interpretation

Inventory loss value (£89.75M) represents 24.8% of estimated gross profit — a material drag on commercial performance, driven more by the operation's scale (2.6M units received) than by an unusually high loss rate. The most significant KPI in this section, however, is dispatch reliability: with only 29.84% of transactions dispatched on time against 40.82% recorded as Delayed, delivery performance is the single weakest area of the operation and the one most likely to affect customer experience and downstream return rates.

9. Warehouse Performance

Warehouse comparisons include both absolute totals and percentage-based measures, since a warehouse with the highest transaction volume does not automatically make it the least efficient warehouse.

9.1 Warehouse Scorecard

Warehouse	Txns	Received	Dispatched	Loss %	Avg Days	On-Time %	Audit Fail %	Gross Profit (£)
Liverpool Logistics	602	307,481	157,676	6.03	5.66	28.07	18.11	43,262,499
Birmingham Distribution	646	338,220	172,244	5.84	5.43	32.66	16.72	45,901,505
Bristol Supply Hub	636	327,289	167,564	5.80	5.68	27.04	17.45	44,888,022
Newcastle Depot	632	331,258	175,908	5.79	5.60	29.43	15.66	49,507,749
London Central Hub	614	314,845	153,148	5.65	5.47	30.46	12.21	38,451,886
Manchester Storage	612	324,524	167,579	5.65	5.49	28.10	13.89	42,786,655
Glasgow Warehouse	627	333,648	177,039	5.58	5.35	33.81	16.43	47,677,762
Leeds Fulfillment	631	337,096	182,717	5.52	5.54	29.00	12.84	49,830,267

9.2 Key Findings

Finding: Highest-Risk Warehouse — Liverpool Logistics

Evidence: Liverpool Logistics recorded the highest inventory loss percentage (6.03%) and the highest audit failure rate (18.11%) of all 8 warehouses, alongside an above-average delivery time of 5.66 days. Business implication: the combination of elevated loss, the weakest compliance record and slower delivery makes this location the clearest single-site risk concentration in the network. Recommendation: prioritise a storage, handling and audit-process review at Liverpool Logistics before the next operating cycle.

Finding: Strongest Overall Performer — Leeds Fulfillment

Evidence: Leeds Fulfillment combines the lowest loss percentage (5.52%) with the highest estimated gross profit (£49.83M) of any warehouse, while handling the second-highest dispatch volume (182,717 units). Business implication: Leeds demonstrates that high throughput and low loss are achievable together, making its storage and handling practices a reasonable benchmark for other sites. Recommendation: document Leeds Fulfillment's current handling and audit procedures as a reference model for underperforming locations.

Finding: Multi-Risk Warehouses (Above-Average Loss AND Above-Average Delivery Time)

Evidence: three warehouses — Liverpool Logistics (6.03% loss, 5.66 days), Bristol Supply Hub (5.80% loss, 5.68 days) and Newcastle Depot (5.79% loss, 5.60 days) — exceed both the overall average loss percentage (5.73%) and the overall average delivery time (5.53 days) simultaneously. Business implication: these three sites combine inventory risk with delivery risk, which compounds the likelihood of customer-facing problems (a delayed shipment that has also sustained damage). Recommendation: treat these three warehouses as the priority group for a combined operational review, rather than addressing loss and delivery issues separately.

Finding: Dispatch Reliability Is Weak Across Every Warehouse

Evidence: even the best-performing warehouse for on-time dispatch, Glasgow Warehouse, achieves only 33.81% on-time delivery; the weakest, Bristol Supply Hub, achieves 27.04%. The spread between the best and

worst warehouse is under 7 percentage points, which is small relative to how far every warehouse sits from full on-time performance. Business implication: dispatch delay is not a location-specific problem — it is systemic across the whole network, suggesting a shared process, carrier, or scheduling constraint rather than a site-management issue. Recommendation: investigate dispatch scheduling and carrier processes at a network level rather than warehouse-by-warehouse.

10. Product and Category Performance

10.1 Category-Level Summary

Category	Txns	Received	Dispatched	Loss %	Return %	Sales Value (£)	Gross Profit (£)
Pharma	1,007	527,791	274,393	5.70	4.53	244,691,702	77,050,747
Electronics	998	540,798	279,298	5.51	4.51	243,922,283	75,832,460
Home Appliances	1,006	522,164	269,066	5.82	4.81	226,023,146	70,334,450
Grocery	999	513,843	267,159	5.72	4.73	226,590,593	69,545,920
Fashion	990	509,765	263,959	5.93	4.64	229,500,909	69,542,768

Finding: Category Risk Profile

Evidence: Fashion carries the highest loss percentage of any category (5.93%) despite ranking only fourth by dispatched volume, while Electronics has the lowest loss percentage (5.51%) despite the highest received volume (540,798 units). Home Appliances has the highest return percentage (4.81%). Business implication: category-level risk is not simply a function of scale — Fashion's higher loss rate and Home Appliances' higher return rate point to product-specific handling or quality issues rather than volume pressure.

Recommendation: review packaging and handling standards for Fashion items, and investigate the drivers of Home Appliances returns (e.g. product fit, damage in transit, or customer expectation mismatches).

10.2 Top Products by Estimated Gross Profit

Product	Category	Gross Profit (£)
Cough Syrup	Pharma	20,626,721
Vitamin Tablets	Pharma	20,245,907
Paracetamol	Pharma	18,781,424
Mixer	Home Appliances	18,546,293
Vacuum Cleaner	Home Appliances	18,169,378
Keyboard	Electronics	17,803,697
Bandages	Pharma	17,396,695

10.3 Highest-Risk Products (Minimum Volume Applied to Avoid Unstable Percentages)

Product	Received Units	Loss %
Jacket	102,834	6.18
T-Shirt	108,887	6.15
Coffee	99,729	6.14
Air Fryer	136,832	6.05
Cereal	101,390	5.94

Products with the highest return percentage (minimum 200 dispatched units applied):

Product	Dispatched Units	Return %
Air Fryer	66,645	5.28
Handbag	51,020	5.19
Rice Pack	55,754	4.94
Smartphone	54,435	4.81
Cereal	53,339	4.81

Finding: Air Fryer Is a Repeat Risk Item

Evidence: Air Fryer appears in both the top-5 highest-loss products (6.05%, well above the 5.73% overall average) and the top-5 highest-return products (5.28%, the single highest return rate of any product in the dataset). Business implication: this product combines physical handling risk with post-dispatch quality or fulfilment risk, making it a priority for root-cause investigation ahead of any category-wide policy change. Recommendation: conduct a focused review of Air Fryer packaging, handling instructions and return reason codes before extending any changes to the rest of the Home Appliances category.

11. Supplier Performance

Supplier comparisons use percentage-based measures throughout, since high-volume suppliers should not be judged on total losses alone.

Supplier	Txns	Received	Loss %	Audit Fail %	Return %	Gross Profit (£)
ASDA Distribution	611	309,674	6.04	14.40	4.60	45,818,653
UK Retail Goods	619	314,432	5.93	16.96	4.81	41,819,605
Tesco Supply Ltd	610	317,507	5.78	16.07	4.48	42,612,594
Prime Wholesale UK	658	348,061	5.74	13.37	4.71	45,460,827
London Trade Corp	616	323,005	5.70	15.58	4.77	48,570,058
Britannia Pharma	630	332,763	5.62	17.62	4.61	46,483,443
Northern Supply Chain	626	333,882	5.55	15.50	4.65	47,083,102
Sainsbury Logistics	630	335,037	5.52	13.97	4.51	44,458,064

Finding: Highest-Risk Supplier — ASDA Distribution

Evidence: ASDA Distribution recorded the highest loss percentage of any supplier (6.04%, against an overall average of 5.73%). Britannia Pharma recorded the highest audit failure rate of any supplier (17.62%). Business implication: these two suppliers warrant separate conversations — ASDA Distribution on physical product quality and packaging, Britannia Pharma on compliance and documentation standards, since their risk profiles differ in nature. Recommendation: schedule a supplier quality review with ASDA Distribution and a compliance/documentation review with Britannia Pharma.

Finding: Priority Supplier — Above-Average Volume AND Above-Average Loss

Evidence: applying the combined filter of above-average supply volume and above-average loss percentage (Handbook task 7.7) identifies exactly one supplier meeting both conditions: Prime Wholesale UK, with the highest received volume of any supplier (348,061 units) and a loss percentage (5.74%) marginally above the 5.73% overall average. Business implication: because Prime Wholesale UK supplies more volume than any other partner, even a small loss-rate premium translates into a large absolute loss exposure, making it the single supplier where a modest quality improvement would have the greatest network-wide impact. Recommendation: prioritise Prime Wholesale UK for supplier engagement, since improvement here scales further than at any other supplier.

12. Operational Performance

12.1 Dispatch Status and Delivery Time

Dispatch Status	Transactions	% of Total	Avg Delivery Days
Delayed	2,041	40.82	8.49
On Time	1,492	29.84	2.01
Slightly Delayed	1,467	29.34	5.00

Finding: Dispatch Delay Is the Operation's Single Largest Risk

Evidence: 40.82% of all transactions are recorded as Delayed (averaging 8.49 delivery days), while only 29.84% are On Time (averaging 2.01 days). This pattern is consistent across every warehouse (best: Glasgow Warehouse at 33.81% on-time; worst: Bristol Supply Hub at 27.04% on-time), every product category and every supplier, indicating the issue is systemic rather than isolated to any single segment. Business implication: this is the largest deviation from an operational ideal found anywhere in the dataset — far larger in scale than the roughly 0.5 percentage-point spread seen in loss rates across warehouses, products and suppliers — and is the finding most likely to affect customer satisfaction, since delayed dispatch is also associated with the highest average delivery time by a wide margin. Recommendation: this should be the number-one priority for operational investigation, ahead of the smaller, segment-specific loss and audit findings elsewhere in this report.

12.2 Employee Handling Analysis

Interpretation Caution

The database shows association, not direct causation. Differences between employees may reflect transaction mix, warehouse conditions, supplier quality or workload rather than individual performance alone. Findings below use wording such as “transactions associated with this employee recorded a higher rate” rather than attributing cause, and recommend investigation of workload, product mix and warehouse conditions rather than direct performance judgement.

Employee	Txns	Loss %	Audit Fail %	Return %
Oliver Smith	644	5.96	15.06	4.59
Charlie Davies	625	5.92	16.16	4.78
Emily Harris	639	5.74	15.81	4.75
George Brown	626	5.74	14.86	4.57
Jack Taylor	580	5.74	16.03	4.76
Amelia Clark	620	5.69	16.13	4.58
Noah Wilson	615	5.56	13.33	4.49
Harry Johnson	651	5.50	15.98	4.61

Workload is evenly distributed across the 8 employees (580–651 transactions each), and loss percentages associated with each employee fall within a narrow 0.46-percentage-point band (5.50%–5.96%). This narrow spread, combined with even workload distribution, does not point to an individual-performance issue; it is more consistent with loss being driven by product, supplier and warehouse conditions shared across the team.

12.3 Storage Section Analysis

Storage Section	Txns	Loss Units	Loss %	Audit Fail %
A1	619	18,574	5.90	15.02
B2	611	18,622	5.87	15.88
D1	668	20,002	5.83	16.77
D2	629	18,964	5.81	15.74
B1	605	18,098	5.73	15.37
C2	625	18,802	5.62	14.24
A2	630	18,500	5.56	13.02
C1	613	18,266	5.54	17.29

Finding: Storage Section C1 Decouples Loss and Audit Risk

Evidence: Storage Section C1 has the lowest loss percentage of any section (5.54%) but the highest audit failure rate (17.29%) — the opposite pattern from Section A1, which has the highest loss percentage (5.90%) but a mid-range audit failure rate (15.02%). Business implication: this indicates that physical inventory loss and audit compliance are driven by different underlying causes in this operation — loss appears more closely tied to product handling, while audit failure may reflect documentation or process-compliance gaps unrelated to physical stock condition. Recommendation: investigate audit procedures specifically at Section C1 (e.g. paperwork completion, scan compliance) separately from the physical handling review recommended for Section A1.

12.4 Temperature-Control Analysis

Temperature Controlled	Txns	Loss %	Audit Fail %	Avg Delivery Days
No	2,469	5.77	15.23	5.56
Yes	2,531	5.69	15.61	5.50

Finding: Temperature Control Shows Minimal Measurable Benefit in This Dataset

Evidence: loss percentage for temperature-controlled inventory (5.69%) is only marginally lower than non-controlled inventory (5.77%), a difference of 0.08 percentage points, and audit failure rate is in fact slightly higher for temperature-controlled inventory (15.61% vs 15.23%). Business implication: on the evidence available, temperature control is not associated with a materially lower loss or higher compliance outcome in this dataset, which may reflect that damage and missing-unit causes in this operation are largely unrelated to temperature sensitivity, or that temperature-control procedures are not being differentiated operationally from standard handling. Recommendation: before further investment in temperature-control infrastructure, verify whether temperature-controlled transactions are being handled under genuinely different procedures, since the data does not currently show a clear performance benefit.

12.5 Stock Audit Analysis

Audit Outcome	Loss %	Return %	Avg Delivery Days
Failed	5.71	4.71	5.53
Passed	5.74	4.63	5.53

Finding: Audit Outcome Shows No Material Link to Inventory Loss in This Dataset

Evidence: transactions with a Failed audit outcome show a loss percentage of 5.71%, nearly identical to Passed transactions at 5.74%, indicating no material descriptive difference between the two groups; average delivery days are also identical (5.53) for both. Business implication: audit failure does not track with higher physical inventory loss in this dataset, which suggests that audit failures may reflect documentation, process or compliance factors not captured in the dataset, rather than the physical condition of stock — though this dataset cannot establish the actual cause. Recommendation: review what specifically causes audits to fail at this business (paperwork, count discrepancies, timing) since the current evidence suggests it is not simply a proxy for damaged or missing stock.

13. Financial Analysis

Financial Terminology

All financial figures in this section are estimates derived from transaction quantities, unit cost and selling price. They represent gross operational values and are described using the terms “estimated sales value”, “dispatched inventory cost”, “estimated gross profit” and “inventory loss value” throughout — never as audited revenue, net profit or realised loss — because transport, labour, storage, tax and other operating expenses are not included in this dataset.

13.1 Overall Financial Position

Metric	Value (GBP)
Total estimated sales value	1,170,728,632.87
Total dispatched inventory cost	808,422,287.40
Total estimated gross profit	362,306,345.47
Total inventory loss value	89,754,534.13
Adjusted contribution estimate (gross profit less loss value)	272,551,811.34
Estimated return value (units returned × selling price)	54,904,102.21

Finding: Inventory Loss Materially Reduces Estimated Contribution

Evidence: total inventory loss value (£89.75M) equals 24.8% of total estimated gross profit (£362.3M); after deducting loss value, the adjusted contribution estimate falls to £272.6M. Business implication: loss reduction is not a marginal efficiency exercise here — it represents roughly a quarter of the commercial upside currently generated. Recommendation: track the adjusted contribution estimate (gross profit less inventory loss value) as a standing KPI alongside gross profit itself, so improvement initiatives are measured against their true bottom-line effect.

13.2 Revenue and Profit by Category

Pharma generates the highest estimated gross profit of any category (£77.05M), narrowly ahead of Electronics (£75.83M), despite Electronics handling a higher dispatched volume. This reflects Pharma's stronger average margin per unit rather than pure volume.

13.3 High-Revenue, Low-Margin Exposure

Task 11.8 identifies transactions with above-average estimated sales value but a margin percentage below 20% — a volume-driven pattern that can appear commercially strong on a sales-value basis while contributing comparatively little profit per unit. The full transaction-level list is provided in the accompanying SQL output (Appendix, Query 11.8) for procurement and pricing review.

14. Time-Series Analysis

The converted `Transaction_Date` field enabled monthly and yearly trend analysis across the full 17-month period (January 2025 to May 2026). Year is grouped together with month throughout, so that the same calendar month appearing in both 2025 and 2026 is never combined incorrectly.

14.1 Overall Pattern

Monthly transaction counts, dispatched volume, loss percentage and estimated gross profit were calculated for all 17 months in the dataset (full detail in Appendix, Queries 12.2–12.7). Because the underlying transaction data is close to uniformly distributed across warehouses, products and suppliers, month-to-month variation in the core rates (loss percentage, return percentage, on-time rate) is modest and does not show a strong seasonal signal within this dataset.

14.2 Year-over-Year Comparison (January–May)

Only January to May exist in both 2025 and 2026, enabling a like-for-like year-over-year comparison for those five months (Handbook task 12.8). The full month-by-month comparison, including percentage change, is provided in the appendix SQL output.

14.3 Month-over-Month Change

LAG and LEAD window functions were used to calculate month-over-month percentage change in dispatch volume, inventory loss value and estimated gross profit (Handbook tasks 12.9, 13.16, 13.17), and a running (cumulative) total of dispatched quantity was calculated using SUM as a window function (task 13.14). These outputs support operational and financial planning by showing the direction and magnitude of recent change without collapsing the underlying monthly detail.

14.4 Peak and Trough Periods

The best- and worst-performing month for dispatch volume, loss percentage and gross profit were identified individually (Handbook task 12.10); the specific calendar months are provided in the appendix SQL output (Query 12.10) for stock, staffing and dispatch planning purposes.

15. Advanced SQL Findings

Advanced SQL techniques — HAVING with conditional aggregation, subqueries, CTEs, window functions and reusable views — were applied only after the descriptive analysis had been validated, to answer questions that a single basic aggregation could not resolve.

15.1 HAVING and Conditional Aggregation

- Warehouses with loss percentage above a 7% threshold: none of the 8 warehouses exceeded this threshold (highest was Liverpool Logistics at 6.03%), indicating no single site represents an extreme outlier.
- Material high-risk products/suppliers (minimum volume applied to avoid unstable percentages from small samples): the filtered list is provided in Appendix Query 13.2, ensuring any product or supplier flagged as high-risk is backed by sufficient transaction volume to be actionable rather than statistical noise.

15.2 Subqueries and CTEs

- A correlated subquery (task 13.5) identified individual transactions with a loss percentage above the overall database average — used as a targeted exception list rather than relying on aggregate rates alone.
- A CTE-based warehouse scorecard (task 13.8) combined volume, loss, delivery and financial measures from four separate CTEs into a single joined result, replicated in the `vw_warehouse_scorecard` view for reuse.
- A multi-factor product-risk CTE (task 13.9) combined loss percentage, return percentage, audit failure rate and delayed percentage into a single `Risk_Factor_Count` per product, giving a combined view of risk that no single metric could show alone.

15.3 Window Functions

- `RANK()` and `DENSE_RANK()` were applied to warehouse loss percentage to demonstrate the two standard tie-handling conventions (task 13.11).
- `PARTITION BY` was used to rank products within each category by dispatched volume, surfacing the top 3 products per category rather than a single dataset-wide ranking that would be dominated by high-volume categories (task 13.12).
- `NTILE(4)` divided all 23 products into profit quartiles, giving a balanced way to discuss “top-quartile” and “bottom-quartile” products regardless of the underlying value distribution (task 13.13).

15.4 Views and Reusable Reporting Layers

Three views were created to support ongoing reporting without repeating calculation logic: `vw_transaction_analytics` (transaction-level metrics), `vw_warehouse_scorecard` (warehouse-level KPIs) and `vw_product_performance` (product-level KPIs). These views underpin the summary tables presented throughout Sections 9–13 of this report and are available for direct reuse in future analysis.

16. Key Findings

The five most important evidence-based conclusions from this analysis are presented below in priority order.

Pri.	Finding	Evidence	Business Impact	Recommendation	KPI to Monitor
1	Dispatch reliability is the operation's weakest area	Only 29.84% on-time dispatch vs 40.82% Delayed, consistent across all 8 warehouses (27.04%–33.81% on-time range)	Far larger deviation from an operational ideal than any loss, audit or return metric; the finding most likely to affect customer experience	Investigate dispatch scheduling and carrier processes at network level, not warehouse-by-warehouse	Overall on-time dispatch rate
2	Liverpool Logistics is the highest combined-risk warehouse	Highest loss % (6.03%), highest audit failure rate (18.11%), above-average delivery time (5.66 days)	Single clearest site-level risk concentration in the network	Prioritise storage, handling and audit-process review at Liverpool Logistics	Warehouse loss %, audit failure rate
3	Inventory loss represents 24.8% of estimated gross profit	£89.75M loss value vs £362.3M gross profit; adjusted contribution estimate £272.6M	Loss reduction has material bottom-line impact, not a marginal efficiency issue	Track adjusted contribution estimate (gross profit less loss value) as a standing KPI	Inventory loss value, adjusted contribution
4	Audit failure shows no material link to higher physical inventory loss	Failed-audit loss % (5.71%) and Passed-audit loss % (5.74%) are nearly identical, indicating no material descriptive difference in this dataset	Audit failures may reflect documentation or process factors not captured in the dataset, rather than physical stock condition	Review specific audit failure causes (paperwork, count discrepancies, timing) separately from loss-reduction initiatives	Audit failure rate, loss %
5	Prime Wholesale UK is the priority supplier for engagement	Highest supply volume (348,061 units) combined with above-average loss % (5.74% vs 5.73% overall)	Because this supplier's volume is largest, even a small quality improvement scales further than at any other supplier	Schedule supplier quality engagement with Prime Wholesale UK	Supplier loss %, supply volume

17. Recommendations

Each recommendation below links directly to a finding in Section 16 and specifies an owner, a measurable target, and the KPI used to confirm whether the action worked.

Action	Owner	Target	KPI Measured	Review Period	Priority
Investigate and reduce systemic dispatch delay across all warehouses	Warehouse Operations / Dispatch	Raise on-time dispatch rate from 29.84% toward an interim target of 45%+	Overall on-time dispatch rate	Next quarterly review	Immediate
Conduct combined storage, handling and audit-process review at Liverpool Logistics, Bristol Supply Hub and Newcastle Depot	Warehouse Operations / Quality	Bring loss percentage at all three sites below the 5.73% network average	Warehouse loss percentage	Next quarterly review	Immediate
Engage Prime Wholesale UK on supplier quality control given its scale	Procurement	Reduce Prime Wholesale UK loss percentage from 5.74% to at or below network average	Supplier loss percentage	Next supplier review cycle	Short-term
Review audit failure root causes separately from physical loss initiatives	Quality / Compliance	Identify whether audit failures are documentation-driven; reduce audit failure rate from 15.42%	Audit failure rate	Next quarterly review	Short-term
Investigate Air Fryer packaging, handling and return reasons as a repeat-risk product	Product / Category Management	Reduce Air Fryer loss % (6.05%) and return % (5.28%) toward category averages	Product-level loss % and return %	Next quarterly review	Short-term
Document Leeds Fulfillment's handling and audit procedures as an internal benchmark	Warehouse Operations	Establish a reusable best-practice reference for underperforming sites	Warehouse loss %, gross profit	Ongoing	Ongoing
Track Adjusted Contribution Estimate (gross profit less inventory loss value) as a standing financial KPI	Finance	Establish visibility of loss's true impact on commercial contribution	Adjusted contribution estimate	Monthly	Ongoing

18. Limitations

This analysis is subject to the following limitations, which should be considered alongside the findings and recommendations above:

- Financial figures are estimates only. Estimated sales value, dispatched inventory cost, estimated gross profit and inventory loss value are derived purely from `Quantity_Dispatched`, `Unit_Cost_GBP` and `Selling_Price_GBP`. They do not include transport, labour, storage, tax, marketing or other operating expenses, and should not be read as audited revenue, net profit or realised financial loss.
- The dataset does not include the causes of returns. `Customer>Returns` records only the quantity returned per transaction; the underlying reason (damage, wrong item, customer preference, quality fault) is not captured, so return-driver conclusions in this report are necessarily inferential (e.g. correlating returns with dispatch or audit status) rather than direct.
- Employee-level findings show association, not causation. As noted in Section 12.2, differences in loss, audit or return rates between employees may reflect transaction mix, warehouse conditions or supplier quality rather than individual performance, and have been presented accordingly.
- No point-in-time stock snapshots are available. `Remaining Stock Units` is a derived transaction-level estimate (received less dispatched, damaged and missing units) rather than a physically verified stock count at any given date, so it should be treated as an operational indicator rather than an audited inventory balance.
- No independent causal evidence is available for any of the associations reported (e.g. temperature control and loss, audit outcome and loss, storage section and audit failure). All such findings describe statistical patterns in the transaction data and are recommended as starting points for operational investigation, not as proven cause-and-effect relationships.
- The dataset covers 17 months (January 2025 to May 2026) with only 5 overlapping months (January–May) available for year-over-year comparison, which limits the strength of seasonal conclusions that can be drawn.

19. Conclusion

Across 5,000 transactions and 17 months of activity, this operation shows a fundamentally sound inventory-quality profile: loss (5.73%), return (4.64%) and pricing-integrity rates are stable, tightly clustered across warehouses, products, suppliers and employees, and unaffected by any material data-quality problems in the underlying dataset. The clearest and most urgent finding, however, is operational rather than physical: dispatch reliability is weak everywhere, with fewer than 3 in 10 transactions dispatched on time and over 4 in 10 recorded as Delayed, consistently across every warehouse in the network.

Secondary priorities are concentrated rather than dataset-wide: Liverpool Logistics, Bristol Supply Hub and Newcastle Depot combine above-average loss with above-average delivery time and warrant a joint operational review; Prime Wholesale UK is the one supplier where its scale means a quality improvement would have outsized network impact; and the disconnect between audit outcome and physical loss suggests the audit process itself, not physical stock handling, needs review.

Immediate management priorities are: (1) a network-wide investigation into dispatch scheduling and delay causes; (2) a combined operational review at the three flagged warehouses; and (3) a supplier engagement with Prime Wholesale UK. Together, these three actions address the findings with the largest measurable gap between current performance and a reasonable operational target, and each is tied to a specific KPI (on-time dispatch rate, warehouse loss percentage, and supplier loss percentage respectively) that can confirm whether the action has worked at the next review cycle.

20. Appendix

20.1 Appendix Query Documentation

The table below documents all 148 individually numbered analytical queries specified in the project handbook, in the same order as they were executed. Each entry lists the handbook task ID, the business question it answers, and the report section where its output is used. The complete, executable SQL code for every query is reproduced in full in Section 20.2 immediately below, and is also provided as a standalone file (warehouse_inventory_analysis.sql) in the accompanying GitHub repository /sql folder, organised in the same order and fully commented.

Query ID	Business Question / Purpose	Report Section
1.1	Display table structure and current data types. Reason: confirms whether numeric, text and date fields imported correctly.	5. Database Preparation
1.2	Display the first 20 rows for a visual check of formats and values.	5. Database Preparation
1.3	Count all records in the table to confirm the expected 5,000 rows loaded.	5. Database Preparation
1.4	Count the number of columns to confirm all 22 expected fields are present.	5. Database Preparation
1.5	List distinct values for each major categorical field.	5. Database Preparation
1.6	Count distinct values for each major categorical field (structural profile).	5. Database Preparation
2.1	Identify duplicate Transaction_ID values.	7. Data Quality Assessment
2.2	Count null values in every column (completeness summary).	7. Data Quality Assessment
2.3	Find blank or whitespace-only values in text columns.	7. Data Quality Assessment
2.4	Identify negative values in quantity, price, delivery or return fields.	7. Data Quality Assessment
2.5	Identify zero values in major quantity and price columns (business-rule review).	7. Data Quality Assessment
2.6	Detect categorical values outside the known lists (unexpected categories).	7. Data Quality Assessment
2.7	Identify records where quantity dispatched exceeds quantity received (potential stock inconsistency).	7. Data Quality Assessment
2.8	Identify records where damaged plus missing units exceed quantity received.	7. Data Quality Assessment
2.9	Identify records where customer returns exceed quantity dispatched.	7. Data Quality Assessment
2.10	Identify records where selling price is below unit cost (loss-making sales).	7. Data Quality Assessment
2.11	Identify invalid delivery days (business rule: 1-10 days is the expected operational range for this operation).	7. Data Quality Assessment
2.12	Identify mismatches between Warehouse_Name and City. A fixed warehouse-city mapping is NOT assumed here: this query profiles how many distinct cities are associated with each warehouse so the relationship can be documented rather than assumed.	7. Data Quality Assessment
2.13	Identify batches associated with multiple products or suppliers (checks whether Batch_Number is a reliable reference).	7. Data Quality Assessment
2.14	Consolidated data-quality summary using conditional aggregation. This summary should appear in the final report before any performance analysis.	7. Data Quality Assessment
3.1	Calculate the total number of transactions.	8. Overall KPI Summary
3.2	Calculate total quantity received and total quantity dispatched.	8. Overall KPI Summary

Query ID	Business Question / Purpose	Report Section
3.3	Calculate total damaged units, missing units and customer returns.	8. Overall KPI Summary
3.4	Calculate average quantity received and average quantity dispatched per transaction.	8. Overall KPI Summary
3.5	Calculate minimum, maximum and average unit cost and selling price.	8. Overall KPI Summary
3.6	Calculate minimum, maximum and average delivery days.	8. Overall KPI Summary
3.7	Calculate total remaining stock units.	8. Overall KPI Summary
3.8	Calculate total inventory loss units and overall loss percentage.	8. Overall KPI Summary
3.9	Calculate total estimated sales value, dispatched inventory cost, estimated gross profit and inventory loss value.	8. Overall KPI Summary
3.10	Calculate overall return percentage, audit failure rate and on-time dispatch rate.	8. Overall KPI Summary
4.1	Compare inbound and outbound transaction counts.	9. Warehouse Performance
4.2	Calculate quantity received and dispatched by inventory type.	9. Warehouse Performance
4.3	Calculate remaining stock for every transaction (transaction-level).	9. Warehouse Performance
4.4	Classify each transaction as high, medium or low remaining stock.	9. Warehouse Performance
4.5	Count transactions with negative, zero and positive remaining stock.	9. Warehouse Performance
4.6	Identify the top 20 transactions by remaining stock (stock accumulation).	9. Warehouse Performance
4.7	Identify the top 20 transactions by inventory loss units (highest-loss transactions).	9. Warehouse Performance
4.8	Compare damage percentage and missing percentage overall.	9. Warehouse Performance
5.1	Count transactions by warehouse (operational workload).	9. Warehouse Performance
5.2	Calculate total quantity received, dispatched and remaining by warehouse.	9. Warehouse Performance
5.3	Calculate average transaction size by warehouse.	9. Warehouse Performance
5.4	Rank warehouses by total quantity dispatched (dispatch-volume ranking).	9. Warehouse Performance
5.5	Calculate the received-to-dispatched ratio by warehouse (inventory conversion).	9. Warehouse Performance
5.6	Calculate damaged units, missing units, total loss units and loss percentage by warehouse.	9. Warehouse Performance
5.7	Calculate inventory loss value by warehouse (financial impact ranking).	9. Warehouse Performance
5.8	Calculate customer returns and return percentage by warehouse.	9. Warehouse Performance
5.9	Calculate on-time, slightly delayed and delayed transaction counts by warehouse.	9. Warehouse Performance
5.10	Calculate average delivery days and on-time dispatch rate by warehouse.	9. Warehouse Performance
5.11	Calculate audit failure count and failure rate by warehouse.	9. Warehouse Performance
5.12	Create a warehouse performance score using selected rates and CASE-based	9. Warehouse

Query ID	Business Question / Purpose	Report Section
	scoring rules (lower loss/failure/delay and higher on-time = better).	Performance
5.13	Identify warehouses with above-average loss percentage AND above-average delivery days (combined inventory and delivery risk).	9. Warehouse Performance
6.1	Count transactions by product category.	10. Product and Category Performance
6.2	Calculate received, dispatched and remaining stock by category.	10. Product and Category Performance
6.3	Calculate total loss units, loss percentage and loss value by category.	10. Product and Category Performance
6.4	Calculate customer returns and return percentage by category.	10. Product and Category Performance
6.5	Calculate estimated sales value and gross profit by category.	10. Product and Category Performance
6.6	Calculate average delivery days and on-time dispatch rate by category.	10. Product and Category Performance
6.7	Compare temperature-controlled and non-temperature-controlled performance within each category.	10. Product and Category Performance
6.8	Identify the top 15 products by dispatched units.	10. Product and Category Performance
6.9	Identify the top 15 products by estimated sales value.	10. Product and Category Performance
6.10	Identify the top 15 products by estimated gross profit.	10. Product and Category Performance
6.11	Identify products with the highest loss percentage (minimum 50 units received across the dataset, to avoid unstable percentages from tiny samples).	10. Product and Category Performance
6.12	Identify products with the highest return percentage (minimum dispatched volume to avoid unstable percentages).	10. Product and Category Performance
6.13	Compare product performance against the average of its own category (over- or under-performance benchmark).	10. Product and Category Performance
6.14	Classify products as high-value/high-risk, high-value/low-risk, low-value/high-risk or low-value/low-risk (portfolio view).	10. Product and Category Performance
7.1	Count transactions and calculate quantity received by supplier.	11. Supplier Performance
7.2	Calculate damaged units, missing units, total loss units and loss percentage by supplier.	11. Supplier Performance
7.3	Calculate inventory loss value by supplier.	11. Supplier Performance
7.4	Calculate audit failure count and failure rate by supplier.	11. Supplier Performance
7.5	Calculate returns and return percentage by supplier.	11. Supplier Performance
7.6	Calculate estimated gross profit by supplier.	11. Supplier Performance
7.7	Identify suppliers with above-average supply volume AND above-average loss percentage (strategically important + elevated risk).	11. Supplier Performance
7.8	Rank suppliers within each product category by loss percentage.	11. Supplier Performance
7.9	Classify suppliers into preferred, monitor and high-risk groups using CASE, based on loss percentage and audit failure rate.	11. Supplier Performance
8.1	Count and calculate the percentage of transactions in each dispatch status.	12. Operational Performance
8.2	Calculate average delivery days for each dispatch status.	12. Operational Performance
8.3	Find the minimum and maximum delivery days within each dispatch status.	12. Operational Performance

Query ID	Business Question / Purpose	Report Section
8.4	Classify delivery days into custom speed bands using CASE.	12. Operational Performance
8.5	Compare dispatch status by warehouse, product category and supplier (segmented dispatch-performance tables).	12. Operational Performance
8.6	Calculate return percentage by dispatch status.	12. Operational Performance
8.7	Calculate estimated sales value and gross profit by dispatch status.	12. Operational Performance
8.8	Identify the top 20 transactions with the longest delivery times.	12. Operational Performance
8.9	Identify warehouses or categories where delayed transactions exceed the overall delayed percentage.	12. Operational Performance
9.1	Count transactions handled by each employee.	12. Operational Performance
9.2	Calculate received and dispatched units handled by each employee.	12. Operational Performance
9.3	Calculate loss units, loss percentage and loss value by employee.	12. Operational Performance
9.4	Calculate average delivery days and dispatch-status distribution by employee.	12. Operational Performance
9.5	Calculate audit failure rate by employee.	12. Operational Performance
9.6	Calculate return percentage by employee.	12. Operational Performance
9.7	Compare each employee with the overall average loss and return rates (benchmark classification, not raw totals).	12. Operational Performance
9.8	Rank employees within each warehouse by handled volume, loss rate and on-time rate (controls for warehouse-environment differences).	12. Operational Performance
10.1	Calculate transaction volume, received quantity and remaining stock by storage section.	12. Operational Performance
10.2	Calculate loss units, loss percentage and loss value by storage section.	12. Operational Performance
10.3	Calculate audit failure rate by storage section.	12. Operational Performance
10.4	Calculate return percentage by storage section.	12. Operational Performance
10.5	Compare transaction volume for temperature-controlled and non-controlled inventory.	12. Operational Performance
10.6	Compare loss percentage, audit failure rate and return percentage by temperature-control status.	12. Operational Performance
10.7	Compare average delivery days and delayed percentage by temperature-control status.	12. Operational Performance
10.8	Analyse temperature-control performance separately by product category (avoids mixing categories with different storage requirements).	12. Operational Performance
10.9	Count passed and failed audits and calculate the failure rate.	12. Operational Performance
10.10	Compare loss percentage between passed and failed audits.	12. Operational Performance
10.11	Compare missing percentage and damage percentage between audit outcomes.	12. Operational Performance

Query ID	Business Question / Purpose	Report Section
10.12	Compare return percentage and delivery days by audit outcome.	12. Operational Performance
10.13	Identify high-value transactions with failed audits (prioritises audit exceptions by financial exposure).	12. Operational Performance
10.14	Calculate total returns and overall return percentage.	12. Operational Performance
10.15	Compare return percentage by warehouse, category, product, supplier and employee (segmented return-rate rankings).	12. Operational Performance
10.16	Identify the top 20 transactions with the highest return percentage.	12. Operational Performance
10.17	Classify return risk using CASE (low, moderate, high).	12. Operational Performance
10.18	Compare returns with dispatch status, audit status and temperature-control status (tests whether returns align with delivery or control issues).	12. Operational Performance
11.1	Calculate estimated sales value, dispatched inventory cost and estimated gross profit for every transaction (transaction-level financial base).	13. Financial Analysis
11.2	Calculate total estimated sales value, cost and gross profit (overall financial KPIs).	13. Financial Analysis
11.3	Calculate unit margin and margin percentage for each product.	13. Financial Analysis
11.4	Calculate revenue and gross profit by warehouse, category, product and supplier (segmented revenue and profit rankings).	13. Financial Analysis
11.5	Calculate inventory loss value by warehouse, category, product, supplier and storage section (segmented loss-value rankings).	13. Financial Analysis
11.6	Estimate the selling value associated with customer returns (gross value exposed to returned units).	13. Financial Analysis
11.7	Calculate profit after deducting inventory loss value as an adjusted analytical measure (shows how stock loss reduces gross operational contribution).	13. Financial Analysis
11.8	Identify high-revenue transactions with low margin percentage (volume-driven transactions that may appear strong but produce weak margins).	13. Financial Analysis
11.9	Identify segments (categories) with high profit contribution AND high loss value (commercially important areas needing risk reduction).	13. Financial Analysis
12.1	Extract year, quarter, month number, month name and day from Transaction_Date (reusable date dimensions).	14. Time-Series Analysis
12.2	Count transactions by year and month (chronological monthly trend).	14. Time-Series Analysis
12.3	Calculate received, dispatched and remaining stock by year and month.	14. Time-Series Analysis
12.4	Calculate loss units, loss percentage and loss value by year and month.	14. Time-Series Analysis
12.5	Calculate return percentage by year and month.	14. Time-Series Analysis
12.6	Calculate on-time rate and average delivery days by year and month.	14. Time-Series Analysis
12.7	Calculate estimated sales value and gross profit by year and month.	14. Time-Series Analysis
12.8	Compare the same month across different years (year-over-year monthly change) - only January to May exist in both 2025 and 2026.	14. Time-Series Analysis
12.9	Calculate month-over-month percentage change for dispatch volume, loss value and gross profit (direction and magnitude of recent change).	14. Time-Series Analysis
12.10	Identify the best and worst month for each major KPI (peak and lowest-performing periods).	14. Time-Series Analysis
13.1	Return warehouses with loss percentage above a selected threshold (7%).	15. Advanced SQL Findings
13.2	Identify products with both sufficient volume (≥ 500 units received) AND high return or loss rates (material high-risk groups, avoiding unstable percentages from very small samples).	15. Advanced SQL Findings

Query ID	Business Question / Purpose	Report Section
13.3	Conditional-aggregation query displaying dispatch statuses as separate columns for each warehouse (compact matrix for reporting).	15. Advanced SQL Findings
13.4	Conditional-aggregation query summarising passed/failed audits by category and by warehouse (audit-status matrix).	15. Advanced SQL Findings
13.5	Subquery to identify transactions with loss percentage above the overall average (above-average loss transactions).	15. Advanced SQL Findings
13.6	Subquery to identify warehouses with average delivery days above the overall warehouse average (below-standard delivery locations).	15. Advanced SQL Findings
13.7	CTE that calculates transaction-level inventory and financial metrics, followed by a grouped analysis (separates calculation logic from final aggregation).	15. Advanced SQL Findings
13.8	Multiple CTEs to calculate warehouse volume, loss, delivery and financial measures before joining them into one complete scorecard.	15. Advanced SQL Findings
13.9	CTE to identify high-risk products based on loss, returns, audit failure and delay measures (multi-factor product-risk table).	15. Advanced SQL Findings
13.10	ROW_NUMBER to assign a unique position to products ordered by gross profit (unique product sequence, even when values are tied).	15. Advanced SQL Findings
13.11	RANK and DENSE_RANK to rank warehouses by loss percentage (alternative ranking treatment for ties).	15. Advanced SQL Findings
13.12	PARTITION BY to rank products within each category by dispatched quantity (top products inside each category).	15. Advanced SQL Findings
13.13	NTILE to divide products into performance quartiles (balanced performance bands based on gross profit).	15. Advanced SQL Findings
13.14	SUM as a window function to calculate cumulative monthly dispatched quantity (running total without collapsing monthly rows).	15. Advanced SQL Findings
13.15	AVG as a window function to compare each warehouse with the average across all warehouses (adds a benchmark beside each result).	15. Advanced SQL Findings
13.16	LAG to compare each month with the previous month (previous-month value and difference).	15. Advanced SQL Findings
13.17	LEAD to compare the current month with the following month (next-month value and difference).	15. Advanced SQL Findings
13.18	View containing transaction-level calculated metrics such as remaining stock, loss units, loss value, sales value and gross profit (reusable transaction analytics view, avoids repeating calculations).	15. Advanced SQL Findings
13.19	Warehouse summary view containing volume, loss, delivery, audit, returns and financial KPIs (stable reporting layer for final analysis).	15. Advanced SQL Findings
13.20	Product performance view with category, volume, loss, returns, revenue and profit measures (reusable product ranking / portfolio view).	15. Advanced SQL Findings

20.2 Final SQL Query Script

The complete, executable SQL script is reproduced below in full, in the exact order it was run, with all comments and query numbering intact. This is the same script provided as `sql/warehouse_inventory_analysis.sql` in the accompanying repository.

```
-- =====
-- WAREHOUSE INVENTORY SQL ANALYSIS
-- Complete Analytical Script
-- Platform: MySQL 8.0
-- Database: any name (create and select it before running this script)
-- =====
-- This script follows the Recommended Query File Structure (Handbook Section 4.3):
-- 0. Database setup and data import (prerequisite, not a handbook task)
-- 1. Database and table inspection
-- 2. Date conversion and validation
-- 3. Data-quality checks
-- 4. Overall KPI analysis
-- 5. Warehouse analysis
-- 6. Product and category analysis
-- 7. Supplier analysis
-- 8. Dispatch and delivery analysis
-- 9. Employee and storage-section analysis
-- 10. Audit, temperature-control and returns analysis
-- 11. Financial analysis
-- 12. Time-series analysis
-- 13. Advanced SQL analysis
-- 14. Final reporting views or summary queries
--
-- Each analytical query is labelled with its handbook task number (e.g. -- 1.1)
-- and preceded by a short comment explaining the business question. Setup
-- steps in Section 0 are deliberately NOT given handbook task numbers, since
-- they are a technical prerequisite rather than an analytical task.
-- =====

-- IMPORTANT: Create and select your target database before running this
-- script, e.g.:
-- CREATE DATABASE warehouse_inventory CHARACTER SET utf8mb4;
-- USE warehouse_inventory;
-- No hardcoded USE statement is included here, so this script can be run
-- against a database of any name without silently switching context.

-- =====
-- SECTION 0: DATABASE SETUP AND DATA IMPORT
-- =====
-- This section makes the script fully self-contained and reproducible from
-- an empty database. It creates the raw staging table exactly as the source
-- CSV is structured (22 columns, Transaction_Date held as text pending
-- conversion in Section 2) and loads data/UK_Warehouse_Inventory_Dataset.csv
-- from the repository. It is safe to re-run: the raw table is dropped and
-- rebuilt from scratch each time, so column counts and row counts are always
-- consistent with a fresh import.

-- Setup Step 1: create the raw staging table (22 columns, matching the
-- source CSV structure before any date conversion or cleaning).
DROP TABLE IF EXISTS warehouse_transactions_raw;

CREATE TABLE warehouse_transactions_raw (
  Transaction_ID          VARCHAR(20)      NOT NULL,
  Transaction_Date_Text   VARCHAR(20)      NOT NULL,    -- original text date, DD-MM-YYYY (maps to the
  CSV's "Transaction_Date" column by position)
  Warehouse_Name          VARCHAR(100)     NOT NULL,
  City                    VARCHAR(100)     NOT NULL,
  Product_Category        VARCHAR(100)     NOT NULL,
  Product_Name            VARCHAR(100)     NOT NULL,
  Supplier_Name           VARCHAR(100)     NOT NULL,
  Batch_Number            VARCHAR(50)      NOT NULL,
  Inventory_Type           VARCHAR(20)      NOT NULL,
  Quantity_Received       INT              NOT NULL,
  Quantity_Dispatched     INT              NOT NULL,
  Damaged_Units           INT              NOT NULL,
  Missing_Units           INT              NOT NULL,
  Unit_Cost_GBP           DECIMAL(10,2)    NOT NULL,
  Selling_Price_GBP       DECIMAL(10,2)    NOT NULL,
  Storage_Section         VARCHAR(10)      NOT NULL,
  Employee_Handling        VARCHAR(100)    NOT NULL,
  Dispatch_Status         VARCHAR(30)      NOT NULL,
  Delivery_Days           INT              NOT NULL,
  Temperature_Controlled   VARCHAR(5)      NOT NULL,
  Stock_Audit_Status      VARCHAR(20)      NOT NULL,
  Customer_Returns        INT              NOT NULL,
  PRIMARY KEY (Transaction_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```

-- Setup Step 2: enable LOCAL INFILE loading for this session (required by
-- some MySQL client/server configurations before LOAD DATA LOCAL INFILE
-- will run). If your server already permits this, the statement is harmless.
SET GLOBAL local_infile = 1;

-- Setup Step 3: load the source dataset. Path is relative to the
-- repository root – adjust it if you run this script from a different
-- working directory or client.
LOAD DATA LOCAL INFILE 'data/UK_Warehouse_Inventory_Dataset.csv'
INTO TABLE warehouse_transactions_raw
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(Transaction_ID, Transaction_Date_Text, Warehouse_Name, City, Product_Category, Product_Name,
Supplier_Name, Batch_Number, Inventory_Type, Quantity_Received, Quantity_Dispatched,
Damaged_Units, Missing_Units, Unit_Cost_GBP, Selling_Price_GBP, Storage_Section,
Employee_Handling, Dispatch_Status, Delivery_Days, Temperature_Controlled,
Stock_Audit_Status, Customer>Returns);

-- Setup Step 4: confirm the import before proceeding to Phase 1.
SELECT COUNT(*) AS Rows_Imported FROM warehouse_transactions_raw;

-- =====
-- SECTION 1: DATABASE AND TABLE INSPECTION (Handbook Phase 1, 5.1)
-- =====

-- 1.1 Display table structure and current data types.
-- Reason: confirms whether numeric, text and date fields imported correctly.
DESCRIBE warehouse_transactions_raw;

-- 1.2 Display the first 20 rows for a visual check of formats and values.
SELECT *
FROM warehouse_transactions_raw
LIMIT 20;

-- 1.3 Count all records in the table to confirm the expected 5,000 rows loaded.
SELECT COUNT(*) AS Total_Row_Count
FROM warehouse_transactions_raw;

-- 1.4 Count the number of columns to confirm all 22 expected fields are present.
SELECT COUNT(*) AS Total_Column_Count
FROM information_schema.columns
WHERE table_schema = DATABASE()
AND table_name = 'warehouse_transactions_raw';

-- 1.5 List distinct values for each major categorical field.
SELECT DISTINCT Warehouse_Name FROM warehouse_transactions_raw ORDER BY Warehouse_Name;
SELECT DISTINCT City FROM warehouse_transactions_raw ORDER BY City;
SELECT DISTINCT Product_Category FROM warehouse_transactions_raw ORDER BY Product_Category;
SELECT DISTINCT Product_Name FROM warehouse_transactions_raw ORDER BY Product_Name;
SELECT DISTINCT Supplier_Name FROM warehouse_transactions_raw ORDER BY Supplier_Name;
SELECT DISTINCT Inventory_Type FROM warehouse_transactions_raw ORDER BY Inventory_Type;
SELECT DISTINCT Dispatch_Status FROM warehouse_transactions_raw ORDER BY Dispatch_Status;
SELECT DISTINCT Stock_Audit_Status FROM warehouse_transactions_raw ORDER BY Stock_Audit_Status;
SELECT DISTINCT Temperature_Controlled FROM warehouse_transactions_raw ORDER BY Temperature_Controlled;

-- 1.6 Count distinct values for each major categorical field (structural profile).
SELECT
COUNT(DISTINCT Warehouse_Name) AS Distinct_Warehouses,
COUNT(DISTINCT City) AS Distinct_Cities,
COUNT(DISTINCT Product_Category) AS Distinct_Categories,
COUNT(DISTINCT Product_Name) AS Distinct_Products,
COUNT(DISTINCT Supplier_Name) AS Distinct_Suppliers,
COUNT(DISTINCT Employee_Handling) AS Distinct_Employees,
COUNT(DISTINCT Storage_Section) AS Distinct_Storage_Sections,
COUNT(DISTINCT Inventory_Type) AS Distinct_Inventory_Types,
COUNT(DISTINCT Dispatch_Status) AS Distinct_Dispatch_Statuses,
COUNT(DISTINCT Stock_Audit_Status) AS Distinct_Audit_Statuses
FROM warehouse_transactions_raw;

-- =====
-- SECTION 2: DATE CONVERSION AND VALIDATION (Handbook 5.2)
-- =====

-- Step 1: Inspect a sample of the original date values to confirm the format
-- is consistently day-month-year (DD-MM-YYYY).
SELECT Transaction_ID, Transaction_Date_Text
FROM warehouse_transactions_raw
LIMIT 20;

-- Step 2: Create a separate DATE column rather than changing the original
-- field immediately (data-safety requirement).
ALTER TABLE warehouse_transactions_raw
DROP COLUMN IF EXISTS Transaction_Date_Converted;

ALTER TABLE warehouse_transactions_raw
ADD COLUMN Transaction_Date_Converted DATE NULL;

```



```

-- Step 3: Parse the text date (DD-MM-YYYY) into the new DATE column.
UPDATE warehouse_transactions_raw
SET Transaction_Date_Converted = STR_TO_DATE(Transaction_Date_Text, '%d-%m-%Y');

-- Step 4: Compare the original text date and converted date side by side
-- for a representative sample.
SELECT Transaction_ID, Transaction_Date_Text, Transaction_Date_Converted
FROM warehouse_transactions_raw
LIMIT 20;

-- Step 5: Validation query - find records where the original date is present
-- but the converted date is null (parsing failure).
SELECT COUNT(*) AS Date_Conversion_Failures
FROM warehouse_transactions_raw
WHERE Transaction_Date_Text IS NOT NULL
  AND Transaction_Date_Text <> ''
  AND Transaction_Date_Converted IS NULL;

-- Step 6: Check the minimum and maximum converted dates and confirm the
-- expected range from 1 January 2025 to 16 May 2026.
SELECT
  MIN(Transaction_Date_Converted) AS Earliest_Transaction_Date,
  MAX(Transaction_Date_Converted) AS Latest_Transaction_Date
FROM warehouse_transactions_raw;

-- Step 7: Build the final validated analytical table. The original text
-- date is archived as Transaction_Date_Text and the converted field is
-- renamed to Transaction_Date, matching the recommended DATE type.
DROP TABLE IF EXISTS warehouse_transactions;

CREATE TABLE warehouse_transactions AS
SELECT
  Transaction_ID,
  Transaction_Date_Converted AS Transaction_Date,
  Transaction_Date_Text, -- original text date retained (data-safety / audit
trail)
  Warehouse_Name,
  City,
  Product_Category,
  Product_Name,
  Supplier_Name,
  Batch_Number,
  Inventory_Type,
  Quantity_Received,
  Quantity_Dispatched,
  Damaged_Units,
  Missing_Units,
  Unit_Cost_GBP,
  Selling_Price_GBP,
  Storage_Section,
  Employee_Handling,
  Dispatch_Status,
  Delivery_Days,
  Temperature_Controlled,
  Stock_Audit_Status,
  Customer>Returns
FROM warehouse_transactions_raw;

ALTER TABLE warehouse_transactions
  ADD PRIMARY KEY (Transaction_ID),
  MODIFY COLUMN Transaction_Date DATE NOT NULL,
  ADD INDEX idx_warehouse (Warehouse_Name),
  ADD INDEX idx_category (Product_Category),
  ADD INDEX idx_supplier (Supplier_Name),
  ADD INDEX idx_date (Transaction_Date);

-- Step 8: Recheck the table structure and confirm Transaction_Date now
-- uses the DATE data type.
DESCRIBE warehouse_transactions;

SELECT COUNT(*) AS Final_Table_Row_Count FROM warehouse_transactions;

-- =====
-- SECTION 3: DATA-QUALITY CHECKS (Handbook Phase 2, 6.1 - 6.3)
-- =====

-- 2.1 Identify duplicate Transaction_ID values.
SELECT Transaction_ID, COUNT(*) AS Occurrence_Count
FROM warehouse_transactions
GROUP BY Transaction_ID
HAVING COUNT(*) > 1;

-- 2.2 Count null values in every column (completeness summary).
SELECT
  SUM(Transaction_ID IS NULL) AS Null_Transaction_ID,
  SUM(Transaction_Date IS NULL) AS Null_Transaction_Date,

```

```

SUM(Warehouse_Name IS NULL)      AS Null_Warehouse_Name,
SUM(City IS NULL)                AS Null_City,
SUM(Product_Category IS NULL)   AS Null_Product_Category,
SUM(Product_Name IS NULL)       AS Null_Product_Name,
SUM(Supplier_Name IS NULL)      AS Null_Supplier_Name,
SUM(Batch_Number IS NULL)       AS Null_Batch_Number,
SUM(Inventory_Type IS NULL)     AS Null_Inventory_Type,
SUM(Quantity_Received IS NULL)  AS Null_Quantity_Received,
SUM(Quantity_Dispatched IS NULL) AS Null_Quantity_Dispatched,
SUM(Damaged_Units IS NULL)      AS Null_Damaged_Units,
SUM(Missing_Units IS NULL)      AS Null_Missing_Units,
SUM(Unit_Cost_GBP IS NULL)      AS Null_Unit_Cost_GBP,
SUM(Selling_Price_GBP IS NULL)  AS Null_Selling_Price_GBP,
SUM(Storage_Section IS NULL)    AS Null_Storage_Section,
SUM(Employee_Handling IS NULL)  AS Null_Employee_Handling,
SUM(Dispatch_Status IS NULL)    AS Null_Dispatch_Status,
SUM(Delivery_Days IS NULL)      AS Null_Delivery_Days,
SUM(Temperature_Controlled IS NULL) AS Null_Temperature_Controlled,
SUM(Stock_Audit_Status IS NULL) AS Null_Stock_Audit_Status,
SUM(Customer_Returns IS NULL)   AS Null_Customer_Returns
FROM warehouse_transactions;

-- 2.3 Find blank or whitespace-only values in text columns.
SELECT
    SUM(TRIM(Warehouse_Name) = '') AS Blank_Warehouse_Name,
    SUM(TRIM(City) = '') AS Blank_City,
    SUM(TRIM(Product_Category) = '') AS Blank_Product_Category,
    SUM(TRIM(Product_Name) = '') AS Blank_Product_Name,
    SUM(TRIM(Supplier_Name) = '') AS Blank_Supplier_Name,
    SUM(TRIM(Batch_Number) = '') AS Blank_Batch_Number,
    SUM(TRIM(Storage_Section) = '') AS Blank_Storage_Section,
    SUM(TRIM(Employee_Handling) = '') AS Blank_Employee_Handling
FROM warehouse_transactions;

-- 2.4 Identify negative values in quantity, price, delivery or return fields.
SELECT COUNT(*) AS Rows_With_Negative_Values
FROM warehouse_transactions
WHERE Quantity_Received < 0 OR Quantity_Dispatched < 0
    OR Damaged_Units < 0    OR Missing_Units < 0
    OR Unit_Cost_GBP < 0    OR Selling_Price_GBP < 0
    OR Delivery_Days < 0    OR Customer_Returns < 0;

-- 2.5 Identify zero values in major quantity and price columns (business-rule review).
SELECT
    SUM(Quantity_Received = 0) AS Zero_Quantity_Received,
    SUM(Quantity_Dispatched = 0) AS Zero_Quantity_Dispatched,
    SUM(Unit_Cost_GBP = 0) AS Zero_Unit_Cost,
    SUM(Selling_Price_GBP = 0) AS Zero_Selling_Price
FROM warehouse_transactions;

-- 2.6 Detect categorical values outside the known lists (unexpected categories).
SELECT DISTINCT Inventory_Type FROM warehouse_transactions
WHERE Inventory_Type NOT IN ('Inbound','Outbound');

SELECT DISTINCT Dispatch_Status FROM warehouse_transactions
WHERE Dispatch_Status NOT IN ('On Time','Slightly Delayed','Delayed');

SELECT DISTINCT Stock_Audit_Status FROM warehouse_transactions
WHERE Stock_Audit_Status NOT IN ('Passed','Failed');

SELECT DISTINCT Temperature_Controlled FROM warehouse_transactions
WHERE Temperature_Controlled NOT IN ('Yes','No');

-- 2.7 Identify records where quantity dispatched exceeds quantity received
-- (potential stock inconsistency).
SELECT Transaction_ID, Quantity_Received, Quantity_Dispatched
FROM warehouse_transactions
WHERE Quantity_Dispatched > Quantity_Received
ORDER BY (Quantity_Dispatched - Quantity_Received) DESC;

-- 2.8 Identify records where damaged plus missing units exceed quantity received.
SELECT Transaction_ID, Quantity_Received, Damaged_Units, Missing_Units,
    (Damaged_Units + Missing_Units) AS Total_Loss_Units
FROM warehouse_transactions
WHERE (Damaged_Units + Missing_Units) > Quantity_Received
ORDER BY Total_Loss_Units DESC;

-- 2.9 Identify records where customer returns exceed quantity dispatched.
SELECT Transaction_ID, Quantity_Dispatched, Customer_Returns
FROM warehouse_transactions
WHERE Customer_Returns > Quantity_Dispatched
ORDER BY (Customer_Returns - Quantity_Dispatched) DESC;

-- 2.10 Identify records where selling price is below unit cost (loss-making sales).
SELECT Transaction_ID, Unit_Cost_GBP, Selling_Price_GBP,
    (Selling_Price_GBP - Unit_Cost_GBP) AS Unit_Margin
FROM warehouse_transactions

```

```

WHERE Selling_Price_GBP < Unit_Cost_GBP
ORDER BY Unit_Margin ASC;

-- 2.11 Identify invalid delivery days (business rule: 1-10 days is the
-- expected operational range for this operation).
SELECT Transaction_ID, Delivery_Days
FROM warehouse_transactions
WHERE Delivery_Days < 1 OR Delivery_Days > 10
ORDER BY Delivery_Days DESC;

-- 2.12 Identify mismatches between Warehouse_Name and City. A fixed
-- warehouse-city mapping is NOT assumed here: this query profiles how many
-- distinct cities are associated with each warehouse so the relationship
-- can be documented rather than assumed.
SELECT
    Warehouse_Name,
    COUNT(DISTINCT City) AS Distinct_Cities_Linked,
    COUNT(*) AS Transaction_Count
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Distinct_Cities_Linked DESC;

-- 2.13 Identify batches associated with multiple products or suppliers
-- (checks whether Batch_Number is a reliable reference).
SELECT
    Batch_Number,
    COUNT(DISTINCT Product_Name) AS Distinct_Products,
    COUNT(DISTINCT Supplier_Name) AS Distinct_Suppliers,
    COUNT(*) AS Transaction_Count
FROM warehouse_transactions
GROUP BY Batch_Number
HAVING COUNT(DISTINCT Product_Name) > 1 OR COUNT(DISTINCT Supplier_Name) > 1
ORDER BY Distinct_Products DESC, Distinct_Suppliers DESC;

-- 2.14 Consolidated data-quality summary using conditional aggregation.
-- This summary should appear in the final report before any performance analysis.
SELECT
    (SELECT COUNT(*) FROM (
        SELECT Transaction_ID FROM warehouse_transactions
        GROUP BY Transaction_ID HAVING COUNT(*) > 1
    ) dup) AS Duplicate_Transaction_IDs,
    SUM(CASE WHEN Quantity_Received IS NULL OR Quantity_Dispatched IS NULL
        OR Unit_Cost_GBP IS NULL OR Selling_Price_GBP IS NULL
        THEN 1 ELSE 0 END) AS Null_Value_Rows,
    SUM(CASE WHEN Quantity_Received < 0 OR Quantity_Dispatched < 0
        OR Damaged_Units < 0 OR Missing_Units < 0
        OR Delivery_Days < 0 OR Customer>Returns < 0
        THEN 1 ELSE 0 END) AS Negative_Value_Rows,
    SUM(CASE WHEN Quantity_Dispatched > Quantity_Received
        OR (Damaged_Units + Missing_Units) > Quantity_Received
        THEN 1 ELSE 0 END) AS Inconsistent_Quantity_Rows,
    SUM(CASE WHEN Customer>Returns > Quantity_Dispatched
        THEN 1 ELSE 0 END) AS Return_Exception_Rows,
    SUM(CASE WHEN Selling_Price_GBP < Unit_Cost_GBP
        THEN 1 ELSE 0 END) AS Pricing_Exception_Rows,
    (SELECT COUNT(*) FROM warehouse_transactions_raw
        WHERE Transaction_Date_Text IS NOT NULL AND Transaction_Date_Text <> ''
        AND Transaction_Date_Converted IS NULL) AS Date_Conversion_Failures
FROM warehouse_transactions;

-- =====
-- SECTION 4: OVERALL KPI ANALYSIS (Handbook Phase 3, 7.1)
-- =====

-- 3.1 Calculate the total number of transactions.
SELECT COUNT(*) AS Total_Transactions
FROM warehouse_transactions;

-- 3.2 Calculate total quantity received and total quantity dispatched.
SELECT
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched
FROM warehouse_transactions;

-- 3.3 Calculate total damaged units, missing units and customer returns.
SELECT
    SUM(Damaged_Units) AS Total_Damaged_Units,
    SUM(Missing_Units) AS Total_Missing_Units,
    SUM(Customer>Returns) AS Total_Customer>Returns
FROM warehouse_transactions;

-- 3.4 Calculate average quantity received and average quantity dispatched
-- per transaction.
SELECT
    ROUND(AVG(Quantity_Received), 2) AS Avg_Received_Per_Transaction,
    ROUND(AVG(Quantity_Dispatched), 2) AS Avg_Dispatched_Per_Transaction
FROM warehouse_transactions;

```

```

-- 3.5 Calculate minimum, maximum and average unit cost and selling price.
SELECT
    MIN(Unit_Cost_GBP)          AS Min_Unit_Cost,
    MAX(Unit_Cost_GBP)          AS Max_Unit_Cost,
    ROUND(AVG(Unit_Cost_GBP), 2) AS Avg_Unit_Cost,
    MIN(Selling_Price_GBP)      AS Min_Selling_Price,
    MAX(Selling_Price_GBP)      AS Max_Selling_Price,
    ROUND(AVG(Selling_Price_GBP), 2) AS Avg_Selling_Price
FROM warehouse_transactions;

-- 3.6 Calculate minimum, maximum and average delivery days.
SELECT
    MIN(Delivery_Days)          AS Min_Delivery_Days,
    MAX(Delivery_Days)          AS Max_Delivery_Days,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM warehouse_transactions;

-- 3.7 Calculate total remaining stock units.
SELECT
    SUM(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Total_Remaining_Stock
FROM warehouse_transactions;

-- 3.8 Calculate total inventory loss units and overall loss percentage.
SELECT
    SUM(Damaged_Units + Missing_Units) AS Total_Inventory_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Overall_Loss_Percentage
FROM warehouse_transactions;

-- 3.9 Calculate total estimated sales value, dispatched inventory cost,
-- estimated gross profit and inventory loss value.
SELECT
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2)          AS
Total_Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * Unit_Cost_GBP), 2)              AS
Total_Dispatched_Inventory_Cost_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Total_Estimated_Gross_Profit_GBP,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS
Total_Inventory_Loss_Value_GBP
FROM warehouse_transactions;

-- 3.10 Calculate overall return percentage, audit failure rate and
-- on-time dispatch rate.
SELECT
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS
Overall_Return_Percentage,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0) * 100, 2) AS
Overall_Audit_Failure_Rate,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0) * 100, 2) AS
Overall_On_Time_Dispatch_Rate
FROM warehouse_transactions;

-- =====
-- SECTION 5: WAREHOUSE ANALYSIS
-- Includes Handbook Phase 4 (Inventory Movement and Stock Position, 8.1)
-- and Phase 5 (Warehouse Performance Analysis, 9.1 - 9.2)
-- =====

-- ---- Phase 4: Inventory Flow Queries ----

-- 4.1 Compare inbound and outbound transaction counts.
SELECT
    Inventory_Type,
    COUNT(*) AS Transaction_Count,
    ROUND(COUNT(*) / (SELECT COUNT(*) FROM warehouse_transactions) * 100, 2) AS Percentage_Of_Total
FROM warehouse_transactions
GROUP BY Inventory_Type
ORDER BY Transaction_Count DESC;

-- 4.2 Calculate quantity received and dispatched by inventory type.
SELECT
    Inventory_Type,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched
FROM warehouse_transactions
GROUP BY Inventory_Type;

-- 4.3 Calculate remaining stock for every transaction (transaction-level).
SELECT
    Transaction_ID,
    Quantity_Received,
    Quantity_Dispatched,
    Damaged_Units,

```

```

    Missing_Units,
    (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Remaining_Stock_Units
FROM warehouse_transactions
ORDER BY Remaining_Stock_Units DESC
LIMIT 50;

```

-- 4.4 Classify each transaction as high, medium or low remaining stock.

```

SELECT
    Transaction_ID,
    (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Remaining_Stock_Units,
    CASE
        WHEN (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) >= 400 THEN 'High'
        WHEN (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) >= 100 THEN 'Medium'
        ELSE 'Low'
    END AS Stock_Level_Classification
FROM warehouse_transactions
ORDER BY Remaining_Stock_Units DESC
LIMIT 50;

```

-- 4.5 Count transactions with negative, zero and positive remaining stock.

```

SELECT
    SUM(CASE WHEN Remaining_Stock_Units < 0 THEN 1 ELSE 0 END) AS Negative_Remaining_Stock_Count,
    SUM(CASE WHEN Remaining_Stock_Units = 0 THEN 1 ELSE 0 END) AS Zero_Remaining_Stock_Count,
    SUM(CASE WHEN Remaining_Stock_Units > 0 THEN 1 ELSE 0 END) AS Positive_Remaining_Stock_Count
FROM (
    SELECT (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS
    Remaining_Stock_Units
    FROM warehouse_transactions
) stock_calc;

```

-- 4.6 Identify the top 20 transactions by remaining stock (stock accumulation).

```

SELECT
    Transaction_ID, Warehouse_Name, Product_Name,
    (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Remaining_Stock_Units
FROM warehouse_transactions
ORDER BY Remaining_Stock_Units DESC
LIMIT 20;

```

-- 4.7 Identify the top 20 transactions by inventory loss units (highest-loss transactions).

```

SELECT
    Transaction_ID, Warehouse_Name, Product_Name,
    (Damaged_Units + Missing_Units) AS Inventory_Loss_Units
FROM warehouse_transactions
ORDER BY Inventory_Loss_Units DESC
LIMIT 20;

```

-- 4.8 Compare damage percentage and missing percentage overall.

```

SELECT
    ROUND(SUM(Damaged_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS Overall_Damage_Percentage,
    ROUND(SUM(Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS Overall_Missing_Percentage
FROM warehouse_transactions;

```

-- ---- Phase 5: Warehouse Performance Analysis ----

-- 5.1 Count transactions by warehouse (operational workload).

```

SELECT
    Warehouse_Name,
    COUNT(*) AS Transaction_Count
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Transaction_Count DESC;

```

-- 5.2 Calculate total quantity received, dispatched and remaining by warehouse.

```

SELECT
    Warehouse_Name,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    SUM(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Total_Remaining_Stock
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Total_Received DESC;

```

-- 5.3 Calculate average transaction size by warehouse.

```

SELECT
    Warehouse_Name,
    ROUND(AVG(Quantity_Received), 2) AS Avg_Received_Per_Transaction,
    ROUND(AVG(Quantity_Dispatched), 2) AS Avg_Dispatched_Per_Transaction
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Avg_Dispatched_Per_Transaction DESC;

```

-- 5.4 Rank warehouses by total quantity dispatched (dispatch-volume ranking).

```

SELECT
    Warehouse_Name,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    RANK() OVER (ORDER BY SUM(Quantity_Dispatched) DESC) AS Dispatch_Volume_Rank
FROM warehouse_transactions

```

```
GROUP BY Warehouse_Name
ORDER BY Dispatch_Volume_Rank;
```

```
-- 5.5 Calculate the received-to-dispatched ratio by warehouse (inventory conversion).
```

```
SELECT
    Warehouse_Name,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    ROUND(SUM(Quantity_Dispatched) / NULLIF(SUM(Quantity_Received), 0), 3) AS Inventory_Conversion_Ratio
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Inventory_Conversion_Ratio DESC;
```

```
-- 5.6 Calculate damaged units, missing units, total loss units and loss
-- percentage by warehouse.
```

```
SELECT
    Warehouse_Name,
    SUM(Damaged_Units) AS Total_Damaged_Units,
    SUM(Missing_Units) AS Total_Missing_Units,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Loss_Percentage DESC;
```

```
-- 5.7 Calculate inventory loss value by warehouse (financial impact ranking).
```

```
SELECT
    Warehouse_Name,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Inventory_Loss_Value_GBP DESC;
```

```
-- 5.8 Calculate customer returns and return percentage by warehouse.
```

```
SELECT
    Warehouse_Name,
    SUM(Customer>Returns) AS Total_Customer>Returns,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Return_Percentage DESC;
```

```
-- 5.9 Calculate on-time, slightly delayed and delayed transaction counts by warehouse.
```

```
SELECT
    Warehouse_Name,
    SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time_Count,
    SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed_Count,
    SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Warehouse_Name;
```

```
-- 5.10 Calculate average delivery days and on-time dispatch rate by warehouse.
```

```
SELECT
    Warehouse_Name,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
On_Time_Dispatch_Rate
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY On_Time_Dispatch_Rate DESC;
```

```
-- 5.11 Calculate audit failure count and failure rate by warehouse.
```

```
SELECT
    Warehouse_Name,
    SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Audit_Failure_Count,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Audit_Failure_Rate DESC;
```

```
-- 5.12 Create a warehouse performance score using selected rates and
```

```
-- CASE-based scoring rules (lower loss/failure/delay and higher on-time = better).
```

```
SELECT
    Warehouse_Name,
    Loss_Percentage,
    Audit_Failure_Rate,
    On_Time_Dispatch_Rate,
    (CASE WHEN Loss_Percentage <= 5 THEN 2 WHEN Loss_Percentage <= 8 THEN 1 ELSE 0 END
    + CASE WHEN Audit_Failure_Rate <= 25 THEN 2 WHEN Audit_Failure_Rate <= 35 THEN 1 ELSE 0 END
    + CASE WHEN On_Time_Dispatch_Rate >= 40 THEN 2 WHEN On_Time_Dispatch_Rate >= 30 THEN 1 ELSE 0 END) AS
Performance_Score,
    CASE
        WHEN (CASE WHEN Loss_Percentage <= 5 THEN 2 WHEN Loss_Percentage <= 8 THEN 1 ELSE 0 END
        + CASE WHEN Audit_Failure_Rate <= 25 THEN 2 WHEN Audit_Failure_Rate <= 35 THEN 1 ELSE 0 END
```

```

        + CASE WHEN On_Time_Dispatch_Rate >= 40 THEN 2 WHEN On_Time_Dispatch_Rate >= 30 THEN 1 ELSE 0
END) >= 5 THEN 'High Performance'
        WHEN (CASE WHEN Loss_Percentage <= 5 THEN 2 WHEN Loss_Percentage <= 8 THEN 1 ELSE 0 END
        + CASE WHEN Audit_Failure_Rate <= 25 THEN 2 WHEN Audit_Failure_Rate <= 35 THEN 1 ELSE 0 END
        + CASE WHEN On_Time_Dispatch_Rate >= 40 THEN 2 WHEN On_Time_Dispatch_Rate >= 30 THEN 1 ELSE 0
END) >= 3 THEN 'Moderate Performance'
        ELSE 'Low Performance'
    END AS Performance_Category
FROM (
    SELECT
        Warehouse_Name,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
        ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100,
2) AS Audit_Failure_Rate,
        ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2)
AS On_Time_Dispatch_Rate
    FROM warehouse_transactions
    GROUP BY Warehouse_Name
) wh_rates
ORDER BY Performance_Score DESC;

-- 5.13 Identify warehouses with above-average loss percentage AND
-- above-average delivery days (combined inventory and delivery risk).
SELECT
    wh.Warehouse_Name,
    wh.Loss_Percentage,
    wh.Avg_Delivery_Days
FROM (
    SELECT
        Warehouse_Name,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
        ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
    FROM warehouse_transactions
    GROUP BY Warehouse_Name
) wh
WHERE wh.Loss_Percentage > (SELECT ROUND(SUM(Damaged_Units + Missing_Units) /
NULLIF(SUM(Quantity_Received), 0) * 100, 2) FROM warehouse_transactions)
    AND wh.Avg_Delivery_Days > (SELECT ROUND(AVG(Delivery_Days), 2) FROM warehouse_transactions)
ORDER BY wh.Loss_Percentage DESC;

-- =====
-- SECTION 6: PRODUCT AND CATEGORY ANALYSIS (Handbook Phase 6, 10.1 - 10.2)
-- =====

-- ---- 10.1 Product Category Analysis ----

-- 6.1 Count transactions by product category.
SELECT
    Product_Category,
    COUNT(*) AS Transaction_Count,
    ROUND(COUNT(*) / (SELECT COUNT(*) FROM warehouse_transactions) * 100, 2) AS Percentage_Of_Total
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Transaction_Count DESC;

-- 6.2 Calculate received, dispatched and remaining stock by category.
SELECT
    Product_Category,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    SUM(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Total_Remaining_Stock
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Total_Received DESC;

-- 6.3 Calculate total loss units, loss percentage and loss value by category.
SELECT
    Product_Category,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Loss_Percentage DESC;

-- 6.4 Calculate customer returns and return percentage by category.
SELECT
    Product_Category,
    SUM(Customer_Returns) AS Total_Customer_Returns,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Return_Percentage DESC;

```

-- 6.5 Calculate estimated sales value and gross profit by category.

```
SELECT
    Product_Category,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Estimated_Gross_Profit_GBP DESC;
```

-- 6.6 Calculate average delivery days and on-time dispatch rate by category.

```
SELECT
    Product_Category,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
On_Time_Dispatch_Rate
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY On_Time_Dispatch_Rate DESC;
```

-- 6.7 Compare temperature-controlled and non-temperature-controlled
-- performance within each category.

```
SELECT
    Product_Category,
    Temperature_Controlled,
    COUNT(*) AS Transaction_Count,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM warehouse_transactions
GROUP BY Product_Category, Temperature_Controlled
ORDER BY Product_Category, Temperature_Controlled;
```

-- ---- 10.2 Product-Level Analysis ----

-- 6.8 Identify the top 15 products by dispatched units.

```
SELECT
    Product_Name,
    Product_Category,
    SUM(Quantity_Dispatched) AS Total_Dispatched
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category
ORDER BY Total_Dispatched DESC
LIMIT 15;
```

-- 6.9 Identify the top 15 products by estimated sales value.

```
SELECT
    Product_Name,
    Product_Category,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category
ORDER BY Estimated_Sales_Value_GBP DESC
LIMIT 15;
```

-- 6.10 Identify the top 15 products by estimated gross profit.

```
SELECT
    Product_Name,
    Product_Category,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category
ORDER BY Estimated_Gross_Profit_GBP DESC
LIMIT 15;
```

-- 6.11 Identify products with the highest loss percentage (minimum 50 units

-- received across the dataset, to avoid unstable percentages from tiny samples).

```
SELECT
    Product_Name,
    Product_Category,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category
HAVING SUM(Quantity_Received) >= 500
ORDER BY Loss_Percentage DESC
LIMIT 15;
```

-- 6.12 Identify products with the highest return percentage

-- (minimum dispatched volume to avoid unstable percentages).

```
SELECT
    Product_Name,
    Product_Category,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
```



```

        SUM(Customer_Returns) AS Total_Customer_Returns,
        ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category
HAVING SUM(Quantity_Dispatched) >= 200
ORDER BY Return_Percentage DESC
LIMIT 15;

-- 6.13 Compare product performance against the average of its own category
-- (over- or under-performance benchmark).
SELECT
    p.Product_Name,
    p.Product_Category,
    p.Product_Loss_Percentage,
    c.Category_Avg_Loss_Percentage,
    ROUND(p.Product_Loss_Percentage - c.Category_Avg_Loss_Percentage, 2) AS Variance_From_Category_Avg,
    CASE WHEN p.Product_Loss_Percentage > c.Category_Avg_Loss_Percentage THEN 'Underperforms Category'
         ELSE 'Outperforms or Matches Category' END AS Benchmark_Result
FROM (
    SELECT Product_Name, Product_Category,
           ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Product_Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Product_Name, Product_Category
) p
JOIN (
    SELECT Product_Category,
           ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Category_Avg_Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Product_Category
) c ON p.Product_Category = c.Product_Category
ORDER BY Variance_From_Category_Avg DESC;

-- 6.14 Classify products as high-value/high-risk, high-value/low-risk,
-- low-value/high-risk or low-value/low-risk (portfolio view).
SELECT
    Product_Name,
    Product_Category,
    Estimated_Gross_Profit_GBP,
    Loss_Percentage,
    CASE
        WHEN Estimated_Gross_Profit_GBP >= (SELECT AVG(gp) FROM (
            SELECT SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)) AS gp
            FROM warehouse_transactions GROUP BY Product_Name) t)
            AND Loss_Percentage >= (SELECT AVG(Damaged_Units + Missing_Units) /
NULLIF(AVG(Quantity_Received), 0) * 100 FROM warehouse_transactions)
        THEN 'High-Value / High-Risk'
        WHEN Estimated_Gross_Profit_GBP >= (SELECT AVG(gp) FROM (
            SELECT SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)) AS gp
            FROM warehouse_transactions GROUP BY Product_Name) t)
        THEN 'High-Value / Low-Risk'
        WHEN Loss_Percentage >= (SELECT AVG(Damaged_Units + Missing_Units) /
NULLIF(AVG(Quantity_Received), 0) * 100 FROM warehouse_transactions)
        THEN 'Low-Value / High-Risk'
        ELSE 'Low-Value / Low-Risk'
    END AS Risk_Value_Classification
FROM (
    SELECT
        Product_Name,
        Product_Category,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Product_Name, Product_Category
) prod_summary
ORDER BY Estimated_Gross_Profit_GBP DESC;

-- =====
-- SECTION 7: SUPPLIER ANALYSIS (Handbook Phase 7, 11.1)
-- =====

-- 7.1 Count transactions and calculate quantity received by supplier.
SELECT
    Supplier_Name,
    COUNT(*) AS Transaction_Count,
    SUM(Quantity_Received) AS Total_Quantity_Received
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Total_Quantity_Received DESC;

-- 7.2 Calculate damaged units, missing units, total loss units and loss
-- percentage by supplier.
SELECT
    Supplier_Name,

```

```

SUM(Damaged_Units) AS Total_Damaged_Units,
SUM(Missing_Units) AS Total_Missing_Units,
SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Loss_Percentage DESC;

-- 7.3 Calculate inventory loss value by supplier.
SELECT
    Supplier_Name,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Inventory_Loss_Value_GBP DESC;

-- 7.4 Calculate audit failure count and failure rate by supplier.
SELECT
    Supplier_Name,
    SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Audit_Failure_Count,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Audit_Failure_Rate DESC;

-- 7.5 Calculate returns and return percentage by supplier.
SELECT
    Supplier_Name,
    SUM(Customer_Returns) AS Total_Customer_Returns,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Return_Percentage DESC;

-- 7.6 Calculate estimated gross profit by supplier.
SELECT
    Supplier_Name,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Estimated_Gross_Profit_GBP DESC;

-- 7.7 Identify suppliers with above-average supply volume AND
-- above-average loss percentage (strategically important + elevated risk).
SELECT
    s.Supplier_Name,
    s.Total_Received,
    s.Loss_Percentage
FROM (
    SELECT
        Supplier_Name,
        SUM(Quantity_Received) AS Total_Received,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Supplier_Name
) s
WHERE s.Total_Received > (SELECT AVG(Total_Received) FROM (
    SELECT SUM(Quantity_Received) AS Total_Received FROM warehouse_transactions GROUP BY Supplier_Name)
t)
AND s.Loss_Percentage > (SELECT ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),
0) * 100, 2) FROM warehouse_transactions)
ORDER BY s.Loss_Percentage DESC;

-- 7.8 Rank suppliers within each product category by loss percentage.
SELECT
    Product_Category,
    Supplier_Name,
    Loss_Percentage,
    RANK() OVER (PARTITION BY Product_Category ORDER BY Loss_Percentage DESC) AS Loss_Rank_Within_Category
FROM (
    SELECT
        Product_Category,
        Supplier_Name,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Product_Category, Supplier_Name
) cat_supplier
ORDER BY Product_Category, Loss_Rank_Within_Category;

-- 7.9 Classify suppliers into preferred, monitor and high-risk groups
-- using CASE, based on loss percentage and audit failure rate.
SELECT
    Supplier_Name,

```

```

    Loss_Percentage,
    Audit_Failure_Rate,
    Return_Percentage,
    CASE
        WHEN Loss_Percentage <= 5 AND Audit_Failure_Rate <= 25 THEN 'Preferred'
        WHEN Loss_Percentage <= 8 AND Audit_Failure_Rate <= 35 THEN 'Monitor'
        ELSE 'High-Risk'
    END AS Supplier_Classification
FROM (
    SELECT
        Supplier_Name,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
        ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100,
2) AS Audit_Failure_Rate,
        ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
    FROM warehouse_transactions
    GROUP BY Supplier_Name
) supp_rates
ORDER BY Loss_Percentage DESC;

-- =====
-- SECTION 8: DISPATCH AND DELIVERY ANALYSIS (Handbook Phase 8, 12.1)
-- =====

-- 8.1 Count and calculate the percentage of transactions in each dispatch status.
SELECT
    Dispatch_Status,
    COUNT(*) AS Transaction_Count,
    ROUND(COUNT(*) / (SELECT COUNT(*) FROM warehouse_transactions) * 100, 2) AS Percentage_Of_Total
FROM warehouse_transactions
GROUP BY Dispatch_Status
ORDER BY Transaction_Count DESC;

-- 8.2 Calculate average delivery days for each dispatch status.
SELECT
    Dispatch_Status,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM warehouse_transactions
GROUP BY Dispatch_Status
ORDER BY Avg_Delivery_Days;

-- 8.3 Find the minimum and maximum delivery days within each dispatch status.
SELECT
    Dispatch_Status,
    MIN(Delivery_Days) AS Min_Delivery_Days,
    MAX(Delivery_Days) AS Max_Delivery_Days
FROM warehouse_transactions
GROUP BY Dispatch_Status
ORDER BY Dispatch_Status;

-- 8.4 Classify delivery days into custom speed bands using CASE.
SELECT
    Transaction_ID,
    Delivery_Days,
    CASE
        WHEN Delivery_Days <= 2 THEN 'Fast'
        WHEN Delivery_Days <= 5 THEN 'Standard'
        WHEN Delivery_Days <= 8 THEN 'Slow'
        ELSE 'Critical'
    END AS Delivery_Speed_Band
FROM warehouse_transactions
ORDER BY Delivery_Days DESC
LIMIT 50;

-- 8.5 Compare dispatch status by warehouse, product category and supplier
-- (segmented dispatch-performance tables).
SELECT
    Warehouse_Name,
    SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time,
    SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed,
    SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Warehouse_Name;

SELECT
    Product_Category,
    SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time,
    SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed,
    SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Product_Category;

SELECT
    Supplier_Name,

```

```

SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time,
SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed,
SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count
FROM warehouse_transactions
GROUP BY Supplier_Name
ORDER BY Supplier_Name;

-- 8.6 Calculate return percentage by dispatch status.
SELECT
    Dispatch_Status,
    SUM(Customer>Returns) AS Total_Customer>Returns,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Dispatch_Status
ORDER BY Return_Percentage DESC;

-- 8.7 Calculate estimated sales value and gross profit by dispatch status.
SELECT
    Dispatch_Status,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY Dispatch_Status
ORDER BY Estimated_Sales_Value_GBP DESC;

-- 8.8 Identify the top 20 transactions with the longest delivery times.
SELECT
    Transaction_ID, Warehouse_Name, Product_Name, Dispatch_Status, Delivery_Days
FROM warehouse_transactions
ORDER BY Delivery_Days DESC
LIMIT 20;

-- 8.9 Identify warehouses or categories where delayed transactions exceed
-- the overall delayed percentage.
SELECT
    Warehouse_Name,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Delayed_Percentage
FROM warehouse_transactions
GROUP BY Warehouse_Name
HAVING ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2)
> (SELECT ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) *
100, 2) FROM warehouse_transactions)
ORDER BY Delayed_Percentage DESC;

SELECT
    Product_Category,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Delayed_Percentage
FROM warehouse_transactions
GROUP BY Product_Category
HAVING ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2)
> (SELECT ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) *
100, 2) FROM warehouse_transactions)
ORDER BY Delayed_Percentage DESC;

-- =====
-- SECTION 9: EMPLOYEE AND STORAGE-SECTION ANALYSIS (Handbook Phase 9, 13.1)
-- =====
-- Note (Interpretation Caution per handbook 13.1): this analysis shows
-- association, not direct causation. Differences may reflect transaction
-- mix, warehouse conditions, supplier quality or workload rather than
-- individual employee performance alone.

-- 9.1 Count transactions handled by each employee.
SELECT
    Employee_Handling,
    COUNT(*) AS Transaction_Count
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Transaction_Count DESC;

-- 9.2 Calculate received and dispatched units handled by each employee.
SELECT
    Employee_Handling,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Total_Received DESC;

-- 9.3 Calculate loss units, loss percentage and loss value by employee.
SELECT
    Employee_Handling,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,

```

```

ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Loss_Percentage DESC;

-- 9.4 Calculate average delivery days and dispatch-status distribution by employee.
SELECT
    Employee_Handling,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
    SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time_Count,
    SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed_Count,
    SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Avg_Delivery_Days;

-- 9.5 Calculate audit failure rate by employee.
SELECT
    Employee_Handling,
    SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Audit_Failure_Count,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Audit_Failure_Rate DESC;

-- 9.6 Calculate return percentage by employee.
SELECT
    Employee_Handling,
    SUM(Customer>Returns) AS Total_Customer>Returns,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Employee_Handling
ORDER BY Return_Percentage DESC;

-- 9.7 Compare each employee with the overall average loss and return rates
-- (benchmark classification, not raw totals).
SELECT
    Employee_Handling,
    Loss_Percentage,
    Return_Percentage,
    CASE WHEN Loss_Percentage > (SELECT ROUND(SUM(Damaged_Units + Missing_Units) /
NULLIF(SUM(Quantity_Received), 0) * 100, 2) FROM warehouse_transactions)
    THEN 'Above-Average Loss' ELSE 'At or Below-Average Loss' END AS Loss_Benchmark,
    CASE WHEN Return_Percentage > (SELECT ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0)
* 100, 2) FROM warehouse_transactions)
    THEN 'Above-Average Returns' ELSE 'At or Below-Average Returns' END AS Return_Benchmark
FROM (
    SELECT
        Employee_Handling,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
        ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
    FROM warehouse_transactions
    GROUP BY Employee_Handling
) emp_rates
ORDER BY Loss_Percentage DESC;

-- 9.8 Rank employees within each warehouse by handled volume, loss rate
-- and on-time rate (controls for warehouse-environment differences).
SELECT
    Warehouse_Name,
    Employee_Handling,
    Total_Handled_Volume,
    RANK() OVER (PARTITION BY Warehouse_Name ORDER BY Total_Handled_Volume DESC) AS
Volume_Rank_Within_Warehouse,
    Loss_Percentage,
    RANK() OVER (PARTITION BY Warehouse_Name ORDER BY Loss_Percentage ASC) AS Loss_Rank_Within_Warehouse,
    On_Time_Rate,
    RANK() OVER (PARTITION BY Warehouse_Name ORDER BY On_Time_Rate DESC) AS OnTime_Rank_Within_Warehouse
FROM (
    SELECT
        Warehouse_Name,
        Employee_Handling,
        SUM(Quantity_Received + Quantity_Dispatched) AS Total_Handled_Volume,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
        ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2)
AS On_Time_Rate
    FROM warehouse_transactions
    GROUP BY Warehouse_Name, Employee_Handling
) wh_emp
ORDER BY Warehouse_Name, Volume_Rank_Within_Warehouse;

-- ---- Storage Section Analysis (Handbook 14.1) ----

```

```

-- 10.1 Calculate transaction volume, received quantity and remaining stock
-- by storage section.
SELECT
    Storage_Section,
    COUNT(*) AS Transaction_Count,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Total_Remaining_Stock
FROM warehouse_transactions
GROUP BY Storage_Section
ORDER BY Storage_Section;

-- 10.2 Calculate loss units, loss percentage and loss value by storage section.
SELECT
    Storage_Section,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Storage_Section
ORDER BY Loss_Percentage DESC;

-- 10.3 Calculate audit failure rate by storage section.
SELECT
    Storage_Section,
    SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Audit_Failure_Count,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate
FROM warehouse_transactions
GROUP BY Storage_Section
ORDER BY Audit_Failure_Rate DESC;

-- 10.4 Calculate return percentage by storage section.
SELECT
    Storage_Section,
    SUM(Customer_Returns) AS Total_Customer_Returns,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Storage_Section
ORDER BY Return_Percentage DESC;

-- =====
-- SECTION 10: AUDIT, TEMPERATURE CONTROL AND RETURNS (Handbook Phase 10)
-- =====

-- ---- 14.2 Temperature-Control Analysis ----

-- 10.5 Compare transaction volume for temperature-controlled and
-- non-controlled inventory.
SELECT
    Temperature_Controlled,
    COUNT(*) AS Transaction_Count,
    SUM(Quantity_Received) AS Total_Received_Units
FROM warehouse_transactions
GROUP BY Temperature_Controlled;

-- 10.6 Compare loss percentage, audit failure rate and return percentage
-- by temperature-control status.
SELECT
    Temperature_Controlled,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Audit_Failure_Rate,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Temperature_Controlled;

-- 10.7 Compare average delivery days and delayed percentage by
-- temperature-control status.
SELECT
    Temperature_Controlled,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
Delayed_Percentage
FROM warehouse_transactions
GROUP BY Temperature_Controlled;

-- 10.8 Analyse temperature-control performance separately by product
-- category (avoids mixing categories with different storage requirements).
SELECT
    Product_Category,
    Temperature_Controlled,
    COUNT(*) AS Transaction_Count,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,

```

```

ROUND(AVG(Delivery_Days), 2)
FROM warehouse_transactions
GROUP BY Product_Category, Temperature_Controlled
ORDER BY Product_Category, Temperature_Controlled;

```

AS Avg_Delivery_Days

```
-- ---- 14.3 Stock Audit Analysis ----
```

```
-- 10.9 Count passed and failed audits and calculate the failure rate.
```

```

SELECT
    Stock_Audit_Status,
    COUNT(*) AS Transaction_Count,
    ROUND(COUNT(*) / (SELECT COUNT(*) FROM warehouse_transactions) * 100, 2) AS Percentage_Of_Total
FROM warehouse_transactions
GROUP BY Stock_Audit_Status;

```

```
-- 10.10 Compare loss percentage between passed and failed audits.
```

```

SELECT
    Stock_Audit_Status,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
FROM warehouse_transactions
GROUP BY Stock_Audit_Status;

```

```
-- 10.11 Compare missing percentage and damage percentage between audit outcomes.
```

```

SELECT
    Stock_Audit_Status,
    ROUND(SUM(Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS Missing_Percentage,
    ROUND(SUM(Damaged_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS Damage_Percentage
FROM warehouse_transactions
GROUP BY Stock_Audit_Status;

```

```
-- 10.12 Compare return percentage and delivery days by audit outcome.
```

```

SELECT
    Stock_Audit_Status,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage,
    ROUND(AVG(Delivery_Days), 2)
FROM warehouse_transactions
GROUP BY Stock_Audit_Status;

```

AS Avg_Delivery_Days

```
-- 10.13 Identify high-value transactions with failed audits
```

```
-- (prioritises audit exceptions by financial exposure).
```

```

SELECT
    Transaction_ID, Warehouse_Name, Product_Name, Supplier_Name,
    ROUND(Quantity_Dispatched * Selling_Price_GBP, 2) AS Estimated_Sales_Value_GBP,
    Stock_Audit_Status
FROM warehouse_transactions
WHERE Stock_Audit_Status = 'Failed'
ORDER BY Estimated_Sales_Value_GBP DESC
LIMIT 20;

```

```
-- ---- 14.4 Customer Return Analysis ----
```

```
-- 10.14 Calculate total returns and overall return percentage.
```

```

SELECT
    SUM(Customer>Returns) AS Total_Customer>Returns,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS
Overall_Return_Percentage
FROM warehouse_transactions;

```

```
-- 10.15 Compare return percentage by warehouse, category, product,
```

```
-- supplier and employee (segmented return-rate rankings).
```

```

SELECT Warehouse_Name,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions GROUP BY Warehouse_Name ORDER BY Return_Percentage DESC;

```

```

SELECT Product_Category,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions GROUP BY Product_Category ORDER BY Return_Percentage DESC;

```

```

SELECT Product_Name,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions GROUP BY Product_Name
HAVING SUM(Quantity_Dispatched) >= 200
ORDER BY Return_Percentage DESC LIMIT 15;

```

```

SELECT Supplier_Name,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions GROUP BY Supplier_Name ORDER BY Return_Percentage DESC;

```

```

SELECT Employee_Handling,
    ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions GROUP BY Employee_Handling ORDER BY Return_Percentage DESC;

```

```
-- 10.16 Identify the top 20 transactions with the highest return percentage.
```

```

SELECT
    Transaction_ID, Warehouse_Name, Product_Name,
    Quantity_Dispatched, Customer>Returns,

```

```

ROUND(Customer_Returns / NULLIF(Quantity_Dispatched, 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
WHERE Quantity_Dispatched > 0
ORDER BY Return_Percentage DESC
LIMIT 20;

-- 10.17 Classify return risk using CASE (low, moderate, high).
SELECT
    Transaction_ID,
    ROUND(Customer_Returns / NULLIF(Quantity_Dispatched, 0) * 100, 2) AS Return_Percentage,
    CASE
        WHEN Customer_Returns / NULLIF(Quantity_Dispatched, 0) * 100 >= 15 THEN 'High'
        WHEN Customer_Returns / NULLIF(Quantity_Dispatched, 0) * 100 >= 5 THEN 'Moderate'
        ELSE 'Low'
    END AS Return_Risk_Category
FROM warehouse_transactions
WHERE Quantity_Dispatched > 0
ORDER BY Return_Percentage DESC
LIMIT 50;

-- 10.18 Compare returns with dispatch status, audit status and
-- temperature-control status (tests whether returns align with delivery
-- or control issues).
SELECT
    Dispatch_Status,
    Stock_Audit_Status,
    Temperature_Controlled,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Dispatch_Status, Stock_Audit_Status, Temperature_Controlled
ORDER BY Return_Percentage DESC;

-- =====
-- SECTION 11: FINANCIAL ANALYSIS (Handbook Phase 11, 15.1)
-- =====
-- Financial measures are estimates derived from transaction quantities,
-- unit cost and selling price. They represent gross operational values and
-- are NOT labelled as net profit, audited revenue or realised loss, since
-- transport, labour, storage, tax and other operating expenses are not
-- included in this dataset.

-- 11.1 Calculate estimated sales value, dispatched inventory cost and
-- estimated gross profit for every transaction (transaction-level financial base).
SELECT
    Transaction_ID,
    ROUND(Quantity_Dispatched * Selling_Price_GBP, 2) AS Estimated_Sales_Value_GBP,
    ROUND(Quantity_Dispatched * Unit_Cost_GBP, 2) AS Dispatched_Inventory_Cost_GBP,
    ROUND(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
ORDER BY Estimated_Gross_Profit_GBP DESC
LIMIT 50;

-- 11.2 Calculate total estimated sales value, cost and gross profit (overall financial KPIs).
SELECT
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS
Total_Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * Unit_Cost_GBP), 2) AS
Total_Dispatched_Inventory_Cost_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Total_Estimated_Gross_Profit_GBP
FROM warehouse_transactions;

-- 11.3 Calculate unit margin and margin percentage for each product.
SELECT
    Product_Name,
    ROUND(AVG(Selling_Price_GBP - Unit_Cost_GBP), 2) AS Avg_Unit_Margin_GBP,
    ROUND(AVG((Selling_Price_GBP - Unit_Cost_GBP) / NULLIF(Selling_Price_GBP, 0)) * 100, 2) AS
Avg_Margin_Percentage
FROM warehouse_transactions
GROUP BY Product_Name
ORDER BY Avg_Margin_Percentage DESC;

-- 11.4 Calculate revenue and gross profit by warehouse, category, product
-- and supplier (segmented revenue and profit rankings).
SELECT Warehouse_Name,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
FROM warehouse_transactions GROUP BY Warehouse_Name ORDER BY Estimated_Gross_Profit_GBP DESC;

SELECT Product_Category,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
FROM warehouse_transactions GROUP BY Product_Category ORDER BY Estimated_Gross_Profit_GBP DESC;

SELECT Product_Name,

```



```

        ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
FROM warehouse_transactions GROUP BY Product_Name ORDER BY Estimated_Gross_Profit_GBP DESC LIMIT 15;

SELECT Supplier_Name,
        ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
FROM warehouse_transactions GROUP BY Supplier_Name ORDER BY Estimated_Gross_Profit_GBP DESC;

-- 11.5 Calculate inventory loss value by warehouse, category, product,
-- supplier and storage section (segmented loss-value rankings).
SELECT Warehouse_Name,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions GROUP BY Warehouse_Name ORDER BY Inventory_Loss_Value_GBP DESC;

SELECT Product_Category,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions GROUP BY Product_Category ORDER BY Inventory_Loss_Value_GBP DESC;

SELECT Product_Name,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions GROUP BY Product_Name ORDER BY Inventory_Loss_Value_GBP DESC LIMIT 15;

SELECT Supplier_Name,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions GROUP BY Supplier_Name ORDER BY Inventory_Loss_Value_GBP DESC;

SELECT Storage_Section,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions GROUP BY Storage_Section ORDER BY Inventory_Loss_Value_GBP DESC;

-- 11.6 Estimate the selling value associated with customer returns
-- (gross value exposed to returned units).
SELECT
        ROUND(SUM(Customer_Returns * Selling_Price_GBP), 2) AS Estimated_Return_Value_GBP
FROM warehouse_transactions;

SELECT
        Product_Category,
        ROUND(SUM(Customer_Returns * Selling_Price_GBP), 2) AS Estimated_Return_Value_GBP
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Estimated_Return_Value_GBP DESC;

-- 11.7 Calculate profit after deducting inventory loss value as an
-- adjusted analytical measure (shows how stock loss reduces gross
-- operational contribution).
SELECT
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP))
        - SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS
Adjusted_Contribution_Estimate_GBP
FROM warehouse_transactions;

SELECT
        Warehouse_Name,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Inventory_Loss_Value_GBP,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP))
        - SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS
Adjusted_Contribution_Estimate_GBP
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Adjusted_Contribution_Estimate_GBP DESC;

-- 11.8 Identify high-revenue transactions with low margin percentage
-- (volume-driven transactions that may appear strong but produce weak margins).
SELECT
        Transaction_ID, Warehouse_Name, Product_Name,
        ROUND(Quantity_Dispatched * Selling_Price_GBP, 2) AS Estimated_Sales_Value_GBP,
        ROUND((Selling_Price_GBP - Unit_Cost_GBP) / NULLIF(Selling_Price_GBP,0) * 100, 2) AS Margin_Percentage
FROM warehouse_transactions
WHERE Quantity_Dispatched * Selling_Price_GBP >= (
        SELECT AVG(Quantity_Dispatched * Selling_Price_GBP) FROM warehouse_transactions)
AND (Selling_Price_GBP - Unit_Cost_GBP) / NULLIF(Selling_Price_GBP,0) * 100 < 20
ORDER BY Estimated_Sales_Value_GBP DESC
LIMIT 20;

-- 11.9 Identify segments (categories) with high profit contribution
-- AND high loss value (commercially important areas needing risk reduction).
SELECT
        Product_Category,

```

```

    Estimated_Gross_Profit_GBP,
    Inventory_Loss_Value_GBP
FROM (
    SELECT
        Product_Category,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS
Inventory_Loss_Value_GBP
    FROM warehouse_transactions
    GROUP BY Product_Category
) cat_fin
WHERE Estimated_Gross_Profit_GBP > (SELECT AVG(gp) FROM (
    SELECT SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)) AS gp
    FROM warehouse_transactions GROUP BY Product_Category) t)
AND Inventory_Loss_Value_GBP > (SELECT AVG(lv) FROM (
    SELECT SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP) AS lv
    FROM warehouse_transactions GROUP BY Product_Category) t2)
ORDER BY Inventory_Loss_Value_GBP DESC;

```

```

-- =====
-- SECTION 12: TIME-SERIES ANALYSIS (Handbook Phase 12, 16.1)
-- =====

```

```

-- Year is always included with month so the same calendar month from
-- different years is never combined incorrectly.

```

```

-- 12.1 Extract year, quarter, month number, month name and day from
-- Transaction_Date (reusable date dimensions).

```

```

SELECT
    Transaction_ID,
    Transaction_Date,
    YEAR(Transaction_Date) AS Txn_Year,
    QUARTER(Transaction_Date) AS Txn_Quarter,
    MONTH(Transaction_Date) AS Txn_Month_Number,
    MONTHNAME(Transaction_Date) AS Txn_Month_Name,
    DAYNAME(Transaction_Date) AS Txn_Day_Name
FROM warehouse_transactions
ORDER BY Transaction_Date
LIMIT 50;

```

```

-- 12.2 Count transactions by year and month (chronological monthly trend).

```

```

SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    COUNT(*) AS Transaction_Count
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

```

```

-- 12.3 Calculate received, dispatched and remaining stock by year and month.

```

```

SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    SUM(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Total_Remaining_Stock
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

```

```

-- 12.4 Calculate loss units, loss percentage and loss value by year and month.

```

```

SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Loss_Value_GBP
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

```

```

-- 12.5 Calculate return percentage by year and month.

```

```

SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

```

```

-- 12.6 Calculate on-time rate and average delivery days by year and month.

```

```

SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100, 2) AS
On_Time_Rate,

```

```

ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

-- 12.7 Calculate estimated sales value and gross profit by year and month.
SELECT
    YEAR(Transaction_Date) AS Txn_Year,
    MONTH(Transaction_Date) AS Txn_Month,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
ORDER BY Txn_Year, Txn_Month;

-- 12.8 Compare the same month across different years (year-over-year
-- monthly change) - only January to May exist in both 2025 and 2026.
SELECT
    MONTH(Transaction_Date) AS Txn_Month,
    MONTHNAME(Transaction_Date) AS Txn_Month_Name,
    SUM(CASE WHEN YEAR(Transaction_Date) = 2025 THEN Quantity_Dispatched ELSE 0 END) AS Dispatched_2025,
    SUM(CASE WHEN YEAR(Transaction_Date) = 2026 THEN Quantity_Dispatched ELSE 0 END) AS Dispatched_2026,
    ROUND(
        (SUM(CASE WHEN YEAR(Transaction_Date) = 2026 THEN Quantity_Dispatched ELSE 0 END)
        - SUM(CASE WHEN YEAR(Transaction_Date) = 2025 THEN Quantity_Dispatched ELSE 0 END))
        / NULLIF(SUM(CASE WHEN YEAR(Transaction_Date) = 2025 THEN Quantity_Dispatched ELSE 0 END), 0) * 100,
    2
    ) AS YoY_Percentage_Change
FROM warehouse_transactions
WHERE MONTH(Transaction_Date) BETWEEN 1 AND 5
GROUP BY MONTH(Transaction_Date), MONTHNAME(Transaction_Date)
ORDER BY Txn_Month;

-- 12.9 Calculate month-over-month percentage change for dispatch volume,
-- loss value and gross profit (direction and magnitude of recent change).
SELECT
    Txn_Year, Txn_Month,
    Total_Dispatched,
    LAG(Total_Dispatched) OVER (ORDER BY Txn_Year, Txn_Month) AS Prev_Month_Dispatched,
    ROUND((Total_Dispatched - LAG(Total_Dispatched) OVER (ORDER BY Txn_Year, Txn_Month))
        / NULLIF(LAG(Total_Dispatched) OVER (ORDER BY Txn_Year, Txn_Month), 0) * 100, 2) AS
Dispatch_MoM_Pct_Change,
    Loss_Value_GBP,
    ROUND((Loss_Value_GBP - LAG(Loss_Value_GBP) OVER (ORDER BY Txn_Year, Txn_Month))
        / NULLIF(LAG(Loss_Value_GBP) OVER (ORDER BY Txn_Year, Txn_Month), 0) * 100, 2) AS
Loss_Value_MoM_Pct_Change,
    Gross_Profit_GBP,
    ROUND((Gross_Profit_GBP - LAG(Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month))
        / NULLIF(LAG(Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month), 0) * 100, 2) AS
Gross_Profit_MoM_Pct_Change
FROM (
    SELECT
        YEAR(Transaction_Date) AS Txn_Year,
        MONTH(Transaction_Date) AS Txn_Month,
        SUM(Quantity_Dispatched) AS Total_Dispatched,
        ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS Loss_Value_GBP,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS Gross_Profit_GBP
    FROM warehouse_transactions
    GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
) monthly
ORDER BY Txn_Year, Txn_Month;

-- 12.10 Identify the best and worst month for each major KPI
-- (peak and lowest-performing periods).
SELECT 'Highest Dispatch Volume' AS Metric, Txn_Year, Txn_Month, Total_Dispatched AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
    SUM(Quantity_Dispatched) Total_Dispatched
    FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value DESC LIMIT 1;

SELECT 'Lowest Dispatch Volume' AS Metric, Txn_Year, Txn_Month, Total_Dispatched AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
    SUM(Quantity_Dispatched) Total_Dispatched
    FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value ASC LIMIT 1;

SELECT 'Highest Loss Percentage' AS Metric, Txn_Year, Txn_Month, Loss_Pct AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) Loss_Pct
    FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value DESC LIMIT 1;

SELECT 'Lowest Loss Percentage' AS Metric, Txn_Year, Txn_Month, Loss_Pct AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) Loss_Pct
    FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value ASC LIMIT 1;

```

```

SELECT 'Highest Gross Profit' AS Metric, Txn_Year, Txn_Month, Gross_Profit AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
      ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) Gross_Profit
      FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value DESC LIMIT 1;

SELECT 'Lowest Gross Profit' AS Metric, Txn_Year, Txn_Month, Gross_Profit AS Value
FROM (SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
      ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) Gross_Profit
      FROM warehouse_transactions GROUP BY 1,2) m
ORDER BY Value ASC LIMIT 1;

-- =====
-- SECTION 13: ADVANCED SQL ANALYSIS (Handbook Phase 13, 17.1 - 17.4)
-- =====

-- ---- 17.1 HAVING and Conditional Aggregation ----

-- 13.1 Return warehouses with loss percentage above a selected threshold (7%).
SELECT
  Warehouse_Name,
  ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage
FROM warehouse_transactions
GROUP BY Warehouse_Name
HAVING ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) > 7
ORDER BY Loss_Percentage DESC;

-- 13.2 Identify products with both sufficient volume (>=500 units received)
-- AND high return or loss rates (material high-risk groups, avoiding
-- unstable percentages from very small samples).
SELECT
  Product_Name,
  SUM(Quantity_Received) AS Total_Received,
  ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
  ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Product_Name
HAVING SUM(Quantity_Received) >= 500
      AND (ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) > 6.5
      OR ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) > 8)
ORDER BY Loss_Percentage DESC;

SELECT
  Supplier_Name,
  SUM(Quantity_Received) AS Total_Received,
  ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
Loss_Percentage,
  ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) AS Return_Percentage
FROM warehouse_transactions
GROUP BY Supplier_Name
HAVING SUM(Quantity_Received) >= 500
      AND (ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) > 6.5
      OR ROUND(SUM(Customer>Returns) / NULLIF(SUM(Quantity_Dispatched), 0) * 100, 2) > 8)
ORDER BY Loss_Percentage DESC;

-- 13.3 Conditional-aggregation query displaying dispatch statuses as
-- separate columns for each warehouse (compact matrix for reporting).
SELECT
  Warehouse_Name,
  SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time,
  SUM(CASE WHEN Dispatch_Status = 'Slightly Delayed' THEN 1 ELSE 0 END) AS Slightly_Delayed,
  SUM(CASE WHEN Dispatch_Status = 'Delayed' THEN 1 ELSE 0 END) AS Delayed_Count,
  COUNT(*) AS Total_Transactions
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Warehouse_Name;

-- 13.4 Conditional-aggregation query summarising passed/failed audits by
-- category and by warehouse (audit-status matrix).
SELECT
  Product_Category,
  SUM(CASE WHEN Stock_Audit_Status = 'Passed' THEN 1 ELSE 0 END) AS Passed,
  SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Failed
FROM warehouse_transactions
GROUP BY Product_Category
ORDER BY Product_Category;

SELECT
  Warehouse_Name,
  SUM(CASE WHEN Stock_Audit_Status = 'Passed' THEN 1 ELSE 0 END) AS Passed,
  SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) AS Failed
FROM warehouse_transactions
GROUP BY Warehouse_Name
ORDER BY Warehouse_Name;

```

-- ---- 17.2 Subqueries and CTEs ----

-- 13.5 Subquery to identify transactions with loss percentage above the
-- overall average (above-average loss transactions).

```
SELECT
    Transaction_ID, Warehouse_Name, Product_Name,
    ROUND((Damaged_Units + Missing_Units) / NULLIF(Quantity_Received, 0) * 100, 2) AS
    Transaction_Loss_Percentage
FROM warehouse_transactions
WHERE (Damaged_Units + Missing_Units) / NULLIF(Quantity_Received, 0) * 100 >
    (SELECT SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100 FROM
    warehouse_transactions)
ORDER BY Transaction_Loss_Percentage DESC
LIMIT 30;
```

-- 13.6 Subquery to identify warehouses with average delivery days above
-- the overall warehouse average (below-standard delivery locations).

```
SELECT
    Warehouse_Name,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM warehouse_transactions
GROUP BY Warehouse_Name
HAVING AVG(Delivery_Days) > (SELECT AVG(Delivery_Days) FROM warehouse_transactions)
ORDER BY Avg_Delivery_Days DESC;
```

-- 13.7 CTE that calculates transaction-level inventory and financial
-- metrics, followed by a grouped analysis (separates calculation logic
-- from final aggregation).

```
WITH Transaction_Metrics AS (
    SELECT
        Transaction_ID,
        Warehouse_Name,
        Product_Category,
        (Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Remaining_Stock_Units,
        (Damaged_Units + Missing_Units) AS Inventory_Loss_Units,
        ROUND((Damaged_Units + Missing_Units) * Unit_Cost_GBP, 2) AS
        Inventory_Loss_Value_GBP,
        ROUND(Quantity_Dispatched * Selling_Price_GBP, 2) AS
        Estimated_Sales_Value_GBP,
        ROUND(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP), 2) AS
        Estimated_Gross_Profit_GBP
    FROM warehouse_transactions
)
SELECT
    Warehouse_Name,
    SUM(Remaining_Stock_Units) AS Total_Remaining_Stock,
    SUM(Inventory_Loss_Units) AS Total_Inventory_Loss_Units,
    ROUND(SUM(Inventory_Loss_Value_GBP), 2) AS Total_Inventory_Loss_Value_GBP,
    ROUND(SUM(Estimated_Sales_Value_GBP), 2) AS Total_Estimated_Sales_Value_GBP,
    ROUND(SUM(Estimated_Gross_Profit_GBP), 2) AS Total_Estimated_Gross_Profit_GBP
FROM Transaction_Metrics
GROUP BY Warehouse_Name
ORDER BY Total_Estimated_Gross_Profit_GBP DESC;
```

-- 13.8 Multiple CTEs to calculate warehouse volume, loss, delivery and
-- financial measures before joining them into one complete scorecard.

```
WITH Volume AS (
    SELECT Warehouse_Name,
        COUNT(*) AS Transaction_Count,
        SUM(Quantity_Received) AS Total_Received,
        SUM(Quantity_Dispatched) AS Total_Dispatched
    FROM warehouse_transactions GROUP BY Warehouse_Name
),
Loss AS (
    SELECT Warehouse_Name,
        SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received), 0) * 100, 2) AS
        Loss_Percentage
    FROM warehouse_transactions GROUP BY Warehouse_Name
),
Delivery AS (
    SELECT Warehouse_Name,
        ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
        ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0) * 100,
        2) AS On_Time_Rate
    FROM warehouse_transactions GROUP BY Warehouse_Name
),
Financials AS (
    SELECT Warehouse_Name,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
        Estimated_Gross_Profit_GBP
    FROM warehouse_transactions GROUP BY Warehouse_Name
)
SELECT
    v.Warehouse_Name,
    v.Transaction_Count, v.Total_Received, v.Total_Dispatched,
```

```

        l.Total_Loss_Units, l.Loss_Percentage,
        d.Avg_Delivery_Days, d.On_Time_Rate,
        f.Estimated_Gross_Profit_GBP
FROM Volume v
JOIN Loss l      ON v.Warehouse_Name = l.Warehouse_Name
JOIN Delivery d  ON v.Warehouse_Name = d.Warehouse_Name
JOIN Financials f ON v.Warehouse_Name = f.Warehouse_Name
ORDER BY f.Estimated_Gross_Profit_GBP DESC;

-- 13.9 CTE to identify high-risk products based on loss, returns, audit
-- failure and delay measures (multi-factor product-risk table).
WITH Product_Risk_Factors AS (
    SELECT
        Product_Name,
        Product_Category,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) AS
Loss_Percentage,
        ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched),0) * 100, 2)          AS
Return_Percentage,
        ROUND(SUM(CASE WHEN Stock_Audit_Status='Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) * 100, 2)
AS Audit_Failure_Rate,
        ROUND(SUM(CASE WHEN Dispatch_Status='Delayed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) * 100, 2)
AS Delayed_Percentage
    FROM warehouse_transactions
    GROUP BY Product_Name, Product_Category
)
SELECT
    Product_Name, Product_Category,
    Loss_Percentage, Return_Percentage, Audit_Failure_Rate, Delayed_Percentage,
    (CASE WHEN Loss_Percentage > 7 THEN 1 ELSE 0 END
+ CASE WHEN Return_Percentage > 8 THEN 1 ELSE 0 END
+ CASE WHEN Audit_Failure_Rate > 32 THEN 1 ELSE 0 END
+ CASE WHEN Delayed_Percentage > 33 THEN 1 ELSE 0 END) AS Risk_Factor_Count
FROM Product_Risk_Factors
ORDER BY Risk_Factor_Count DESC, Loss_Percentage DESC
LIMIT 20;

-- ---- 17.3 Window Functions ----

-- 13.10 ROW_NUMBER to assign a unique position to products ordered by
-- gross profit (unique product sequence, even when values are tied).
SELECT
    Product_Name,
    Estimated_Gross_Profit_GBP,
    ROW_NUMBER() OVER (ORDER BY Estimated_Gross_Profit_GBP DESC) AS Product_Sequence
FROM (
    SELECT Product_Name,
        ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
    FROM warehouse_transactions
    GROUP BY Product_Name
) p;

-- 13.11 RANK and DENSE_RANK to rank warehouses by loss percentage
-- (alternative ranking treatment for ties).
SELECT
    Warehouse_Name,
    Loss_Percentage,
    RANK()      OVER (ORDER BY Loss_Percentage DESC) AS Loss_Rank,
    DENSE_RANK() OVER (ORDER BY Loss_Percentage DESC) AS Loss_Dense_Rank
FROM (
    SELECT Warehouse_Name,
        ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) AS
Loss_Percentage
    FROM warehouse_transactions
    GROUP BY Warehouse_Name
) w
ORDER BY Loss_Rank;

-- 13.12 PARTITION BY to rank products within each category by dispatched
-- quantity (top products inside each category).
SELECT *
FROM (
    SELECT
        Product_Category,
        Product_Name,
        Total_Dispatched,
        RANK() OVER (PARTITION BY Product_Category ORDER BY Total_Dispatched DESC) AS Rank_Within_Category
    FROM (
        SELECT Product_Category, Product_Name,
            SUM(Quantity_Dispatched) AS Total_Dispatched
        FROM warehouse_transactions
        GROUP BY Product_Category, Product_Name
    ) pc
) ranked
WHERE Rank_Within_Category <= 3
ORDER BY Product_Category, Rank_Within_Category;

```

-- 13.13 NTILE to divide products into performance quartiles
 -- (balanced performance bands based on gross profit).

```
SELECT
  Product_Name,
  Estimated_Gross_Profit_GBP,
  NTILE(4) OVER (ORDER BY Estimated_Gross_Profit_GBP DESC) AS Profit_Quartile
FROM (
  SELECT Product_Name,
         ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
    FROM warehouse_transactions
   GROUP BY Product_Name
) p
ORDER BY Profit_Quartile, Estimated_Gross_Profit_GBP DESC;
```

-- 13.14 SUM as a window function to calculate cumulative monthly
 -- dispatched quantity (running total without collapsing monthly rows).

```
SELECT
  Txn_Year, Txn_Month, Monthly_Dispatched,
  SUM(Monthly_Dispatched) OVER (ORDER BY Txn_Year, Txn_Month) AS Running_Total_Dispatched
FROM (
  SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
         SUM(Quantity_Dispatched) AS Monthly_Dispatched
    FROM warehouse_transactions
   GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
) m
ORDER BY Txn_Year, Txn_Month;
```

-- 13.15 AVG as a window function to compare each warehouse with the
 -- average across all warehouses (adds a benchmark beside each result).

```
SELECT
  Warehouse_Name,
  Total_Dispatched,
  ROUND(AVG(Total_Dispatched) OVER (), 2) AS Overall_Warehouse_Avg_Dispatched,
  ROUND(Total_Dispatched - AVG(Total_Dispatched) OVER (), 2) AS Variance_From_Avg
FROM (
  SELECT Warehouse_Name, SUM(Quantity_Dispatched) AS Total_Dispatched
    FROM warehouse_transactions
   GROUP BY Warehouse_Name
) w
ORDER BY Variance_From_Avg DESC;
```

-- 13.16 LAG to compare each month with the previous month
 -- (previous-month value and difference).

```
SELECT
  Txn_Year, Txn_Month, Monthly_Gross_Profit_GBP,
  LAG(Monthly_Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month) AS Prev_Month_Gross_Profit_GBP,
  ROUND(Monthly_Gross_Profit_GBP - LAG(Monthly_Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month), 2) AS
Difference_From_Prev_Month
FROM (
  SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
         ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Monthly_Gross_Profit_GBP
    FROM warehouse_transactions
   GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
) m
ORDER BY Txn_Year, Txn_Month;
```

-- 13.17 LEAD to compare the current month with the following month
 -- (next-month value and difference).

```
SELECT
  Txn_Year, Txn_Month, Monthly_Gross_Profit_GBP,
  LEAD(Monthly_Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month) AS Next_Month_Gross_Profit_GBP,
  ROUND(LEAD(Monthly_Gross_Profit_GBP) OVER (ORDER BY Txn_Year, Txn_Month) - Monthly_Gross_Profit_GBP, 2) AS
Difference_To_Next_Month
FROM (
  SELECT YEAR(Transaction_Date) Txn_Year, MONTH(Transaction_Date) Txn_Month,
         ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Monthly_Gross_Profit_GBP
    FROM warehouse_transactions
   GROUP BY YEAR(Transaction_Date), MONTH(Transaction_Date)
) m
ORDER BY Txn_Year, Txn_Month;
```

-- ---- 17.4 Views and Reusable Reporting Layers ----

-- 13.18 View containing transaction-level calculated metrics such as
 -- remaining stock, loss units, loss value, sales value and gross profit
 -- (reusable transaction analytics view, avoids repeating calculations).

CREATE OR REPLACE VIEW vw_transaction_analytics AS

```
SELECT
  Transaction_ID,
  Transaction_Date,
  Warehouse_Name,
  City,
  Product_Category,
```

```

Product_Name,
Supplier_Name,
Inventory_Type,
Dispatch_Status,
Stock_Audit_Status,
Temperature_Controlled,
Quantity_Received,
Quantity_Dispatched,
Damaged_Units,
Missing_Units,
Customer_Returns,
Delivery_Days,
(Quantity_Received - Quantity_Dispatched - Damaged_Units - Missing_Units) AS Remaining_Stock_Units,
(Damaged_Units + Missing_Units) AS Inventory_Loss_Units,
ROUND((Damaged_Units + Missing_Units) / NULLIF(Quantity_Received,0) * 100, 2) AS
Inventory_Loss_Percentage,
ROUND(Customer_Returns / NULLIF(Quantity_Dispatched,0) * 100, 2) AS Return_Percentage,
ROUND(Quantity_Dispatched * Selling_Price_GBP, 2) AS Estimated_Sales_Value_GBP,
ROUND(Quantity_Dispatched * Unit_Cost_GBP, 2) AS
Dispatched_Inventory_Cost_GBP,
ROUND(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP), 2) AS Estimated_Gross_Profit_GBP,
ROUND((Damaged_Units + Missing_Units) * Unit_Cost_GBP, 2) AS Inventory_Loss_Value_GBP
FROM warehouse_transactions;

SELECT * FROM vw_transaction_analytics LIMIT 10;

-- 13.19 Warehouse summary view containing volume, loss, delivery, audit,
-- returns and financial KPIs (stable reporting layer for final analysis).
CREATE OR REPLACE VIEW vw_warehouse_scorecard AS
SELECT
    Warehouse_Name,
    COUNT(*) AS Transaction_Count,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) AS
Loss_Percentage,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days,
    ROUND(SUM(CASE WHEN Dispatch_Status = 'On Time' THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) * 100, 2) AS
On_Time_Dispatch_Rate,
    ROUND(SUM(CASE WHEN Stock_Audit_Status = 'Failed' THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) * 100, 2) AS
Audit_Failure_Rate,
    SUM(Customer_Returns) AS
Total_Customer_Returns,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched),0) * 100, 2) AS Return_Percentage,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS
Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP,
    ROUND(SUM((Damaged_Units + Missing_Units) * Unit_Cost_GBP), 2) AS
Inventory_Loss_Value_GBP
FROM warehouse_transactions
GROUP BY Warehouse_Name;

SELECT * FROM vw_warehouse_scorecard ORDER BY Estimated_Gross_Profit_GBP DESC;

-- 13.20 Product performance view with category, volume, loss, returns,
-- revenue and profit measures (reusable product ranking / portfolio view).
CREATE OR REPLACE VIEW vw_product_performance AS
SELECT
    Product_Name,
    Product_Category,
    COUNT(*) AS Transaction_Count,
    SUM(Quantity_Received) AS Total_Received,
    SUM(Quantity_Dispatched) AS Total_Dispatched,
    SUM(Damaged_Units + Missing_Units) AS Total_Loss_Units,
    ROUND(SUM(Damaged_Units + Missing_Units) / NULLIF(SUM(Quantity_Received),0) * 100, 2) AS
Loss_Percentage,
    SUM(Customer_Returns) AS
Total_Customer_Returns,
    ROUND(SUM(Customer_Returns) / NULLIF(SUM(Quantity_Dispatched),0) * 100, 2) AS Return_Percentage,
    ROUND(SUM(Quantity_Dispatched * Selling_Price_GBP), 2) AS
Estimated_Sales_Value_GBP,
    ROUND(SUM(Quantity_Dispatched * (Selling_Price_GBP - Unit_Cost_GBP)), 2) AS
Estimated_Gross_Profit_GBP
FROM warehouse_transactions
GROUP BY Product_Name, Product_Category;

SELECT * FROM vw_product_performance ORDER BY Estimated_Gross_Profit_GBP DESC LIMIT 15;

-- =====
-- SECTION 14: FINAL REPORTING VIEWS / SUMMARY QUERIES
-- =====
-- The three views above (vw_transaction_analytics, vw_warehouse_scorecard,
-- vw_product_performance) serve as the final reusable reporting layer.
-- The summary query below draws them together into one master KPI table
-- used directly in the Executive Summary of the analysis report.

```



```

SELECT
  (SELECT COUNT(*) FROM warehouse_transactions) AS
Total_Transactions,
  (SELECT SUM(Quantity_Received) FROM warehouse_transactions) AS
Total_Received,
  (SELECT SUM(Quantity_Dispatched) FROM warehouse_transactions) AS
Total_Dispatched,
  (SELECT ROUND(SUM(Damaged_Units+Missing_Units)/NULLIF(SUM(Quantity_Received),0)*100,2)
   FROM warehouse_transactions) AS
Overall_Loss_Percentage,
  (SELECT ROUND(SUM(Customer>Returns)/NULLIF(SUM(Quantity_Dispatched),0)*100,2)
   FROM warehouse_transactions) AS
Overall_Return_Percentage,
  (SELECT ROUND(SUM(CASE WHEN Stock_Audit_Status='Failed' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)*100,2)
   FROM warehouse_transactions) AS
Overall_Audit_Failure_Rate,
  (SELECT ROUND(SUM(CASE WHEN Dispatch_Status='On Time' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)*100,2)
   FROM warehouse_transactions) AS
Overall_On_Time_Rate,
  (SELECT ROUND(SUM(Quantity_Dispatched*Selling_Price_GBP),2) FROM warehouse_transactions) AS
Total_Estimated_Sales_Value_GBP,
  (SELECT ROUND(SUM(Quantity_Dispatched*(Selling_Price_GBP-Unit_Cost_GBP)),2)
   FROM warehouse_transactions) AS
Total_Estimated_Gross_Profit_GBP,
  (SELECT ROUND(SUM((Damaged_Units+Missing_Units)*Unit_Cost_GBP),2) FROM warehouse_transactions) AS
Total_Inventory_Loss_Value_GBP;

-- =====
-- END OF SCRIPT
-- =====

```

20.3 Data Dictionary

Column	Type	Purpose
Transaction_ID	VARCHAR (Primary Key)	Unique transaction reference
Transaction_Date	DATE (converted)	Transaction timing; used for all time-series analysis
Warehouse_Name	VARCHAR	Warehouse comparison
City	VARCHAR	Geographic / destination field
Product_Category	VARCHAR	High-level product grouping
Product_Name	VARCHAR	Product-level analysis
Supplier_Name	VARCHAR	Supplier quality and volume analysis
Batch_Number	VARCHAR	Batch traceability
Inventory_Type	VARCHAR	Inbound / Outbound classification
Quantity_Received	INT	Inflow base measure
Quantity_Dispatched	INT	Outflow base measure
Damaged_Units	INT	Quality / financial-loss analysis
Missing_Units	INT	Shrinkage / operational-control analysis
Unit_Cost_GBP	DECIMAL(10,2)	Cost and margin calculations
Selling_Price_GBP	DECIMAL(10,2)	Revenue and profit calculations
Storage_Section	VARCHAR	Storage-location risk analysis
Employee_Handling	VARCHAR	Workload and exception analysis
Dispatch_Status	VARCHAR	Delivery-performance category
Delivery_Days	INT	Delivery-time analysis
Temperature_Controlled	VARCHAR	Storage-requirement analysis
Stock_Audit_Status	VARCHAR	Compliance / audit-risk analysis
Customer>Returns	INT	Post-dispatch quality analysis

20.4 Core Metric Definitions

Metric	Definition
Remaining Stock Units	Quantity Received – Quantity Dispatched – Damaged Units – Missing Units
Inventory Loss Units	Damaged Units + Missing Units
Inventory Loss Percentage	Inventory Loss Units ÷ Quantity Received × 100
Damage Percentage	Damaged Units ÷ Quantity Received × 100
Missing Percentage	Missing Units ÷ Quantity Received × 100
Return Percentage	Customer Returns ÷ Quantity Dispatched × 100
Estimated Sales Value	Quantity Dispatched × Selling Price
Dispatched Inventory Cost	Quantity Dispatched × Unit Cost
Estimated Gross Profit	Quantity Dispatched × (Selling Price – Unit Cost)
Inventory Loss Value	Inventory Loss Units × Unit Cost
Audit Failure Rate	Failed Audits ÷ Total Audits × 100
On-Time Dispatch Rate	On-Time Transactions ÷ All Dispatch-Status Records × 100